

RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution, Affiliated to Anna
University Rajalakshmi Nagar, Thandalam – 602-
105



DEPARTMENT OF INFORMATION TECHNOLOGY

IT23431 - MONGODB ESSENTIALS

(Regulation 2023)

LAB MANUAL

Name	: PRATUL V
Register No.	: 2116241001174
Year / Branch / Section	: II / B-TECH / C
Semester	: IV
Academic Year	: 2025-2026

Course Code	Course Title (Laboratory Integrated Theory Course)	L	T	P	C
IT23431	MongoDB Essentials	2	0	2	3

LIST OF EXPERIMENTS			
SNO	NAME OF THE EXPERIMENT	PAGE NO	SIGN
1	MongoDB installation and basic setup		
2	Creating and dropping tables with various constraints		
3	Performing CRUD Operations A] Vehicle Details B] Maintenance Details C] Service Records		
4	A]Projection B]Aggregate		
5	Data Modeling		
6	Perform Schema Design and Indexing		
7	Building aggregation pipelines		
8	Performing Data Transformation		
9	Establishing connection between MongoDB and java		
10	Develop a CRUD Application using MongoDB(Mini Project)		

Ex. Nos. 1	MONGO DB INSTALLATION AND BASIC SET UP
Date :	

Aim:

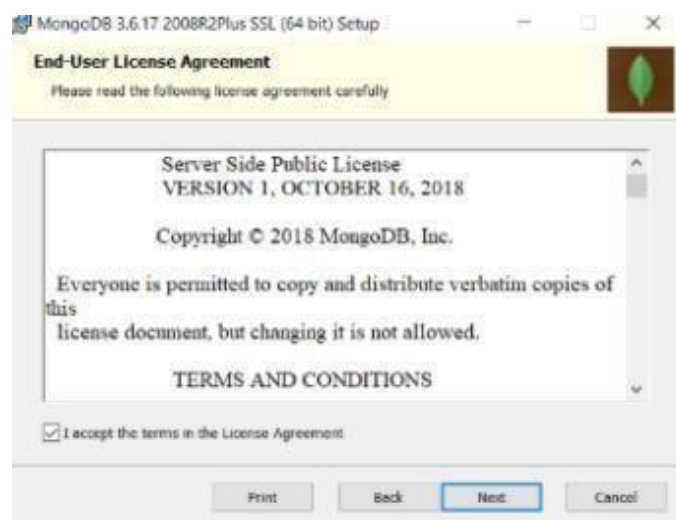
To study the installation of MongoDB setup environment.

Procedure for installation:

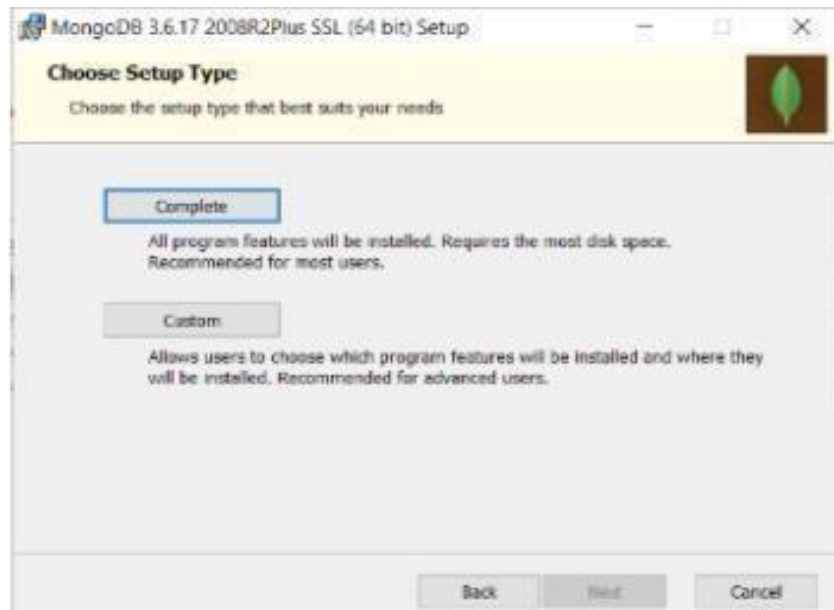
1. Download the installer.
2. Link : <https://www.mongodb.com/docs/manual/installation/>
3. Run the installer.



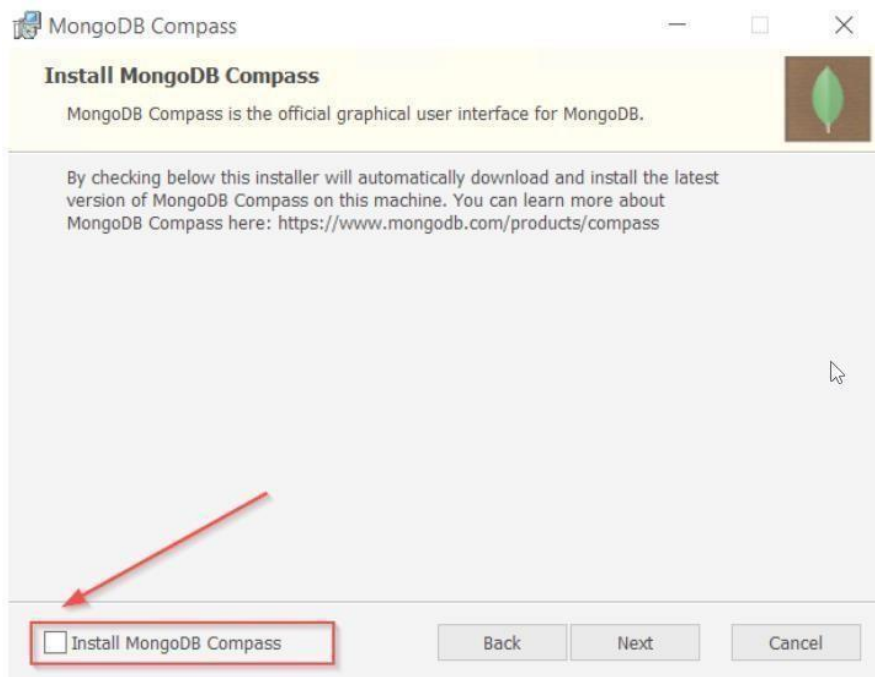
4. Accept the terms in the License agreement and click out.



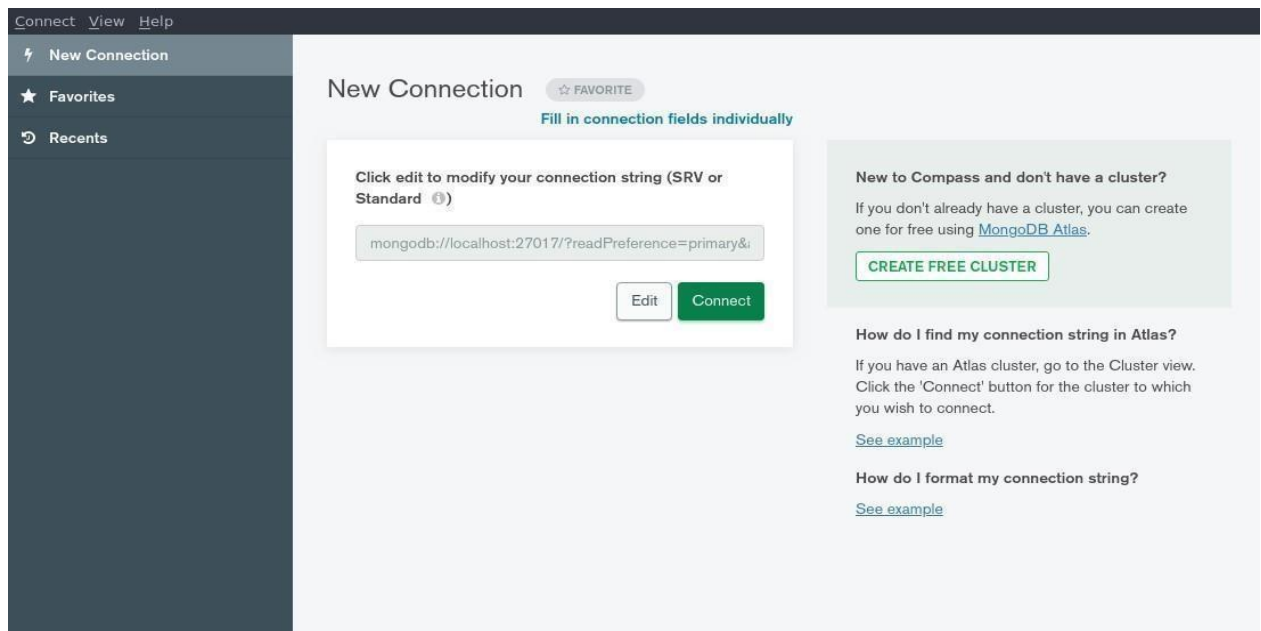
5. Choose the setup type as Complete and click next.



6. Install MongoDB Compass.



7. Click next twice.
8. Click Install and finish.



9. Connect the database.
10. The installation is complete and basic set up is done.

Result:

Thus the MongoDB installation and basic setup environment is successfully complete.

Ex. No. 2	CREATING AND DROPPING TABLES WITH VARIOUS CONSTRAINTS
Date:	

Aim:

To create and drop collection with various constraints.

Procedure:

CREATE

1. Creating a database using the Use <db> command: syntax :

use <databaseName>

Query: use

fmcgSalesTracker Output:

>switch to fmcgSalesTracker

If fmcgSalesTracker does not exist, MongoDB will create it when data is inserted.

2. Creating a collection using the db.createCollection("<collectionName>") command :

The db.createCollection("<collectionName>") command in MongoDB creates a new collection with the specified name in the current database.

syntax : db.createCollection("<collectionName>")

Query: db.createCollection("salesData")

Output:

>{ ok :1}

.DROP

Dropping a collection using db.<collectionName>.drop() the command:

The db.<collectionName>.drop() command in MongoDB permanently

deletes the specified collection and all its documents from the database

syntax : `db.<collectionName>.drop()`

Query: `db.salesData.drop()`

Output

`< true`

Dropping a database using the `db.dropDatabase()` command:

The `db.dropDatabase()` command is used to permanently delete the selected database along with all its collections and documents.

Before executing, ensure you have switched to the correct database using the `use database_name` command.

syntax : `db.dropDatabase()`

Query:

`> use fmcgSalesTracker`

`> db.dropDatabase()`

Output:

`> switched to db fmcgSalesTracker`

`> { ok : 1, dropped : 'fmcgSalesTracker' }`

Result:

Thus the creating and dropping database and collection on MongoDB compass was execute successfully.

Ex. No. 3	CRUD OPERATIONS
Date:	

Aim:

To program CRUD operation on three collection in MongoDB Compass.

PROCEDURE:

Create Collection “salesData”:

```
> db.createCollection("salesData")
```

Create:

InsertOne:

Query:

```
db.salesData.insertOne({
  productName: "Dove Shampoo",
  productNumber: "PRD1001", brand:
  "Dove", category: "Personal Care"
});
```

Output:

```
{
  "acknowledged": true,
  "insertedId": ObjectId("67f6a55013545e3b0785613a")
}
```

InsertMany:

Query:

```
db.salesData.insertMany([
  {
    productName: "Maggi Noodles",
    productNumber: "PRD1002",
    brand: "Nestle",
```



```
category: "Food"
},
{
productName: "Surf Excel",
productNumber: "PRD1003",
brand: "HUL", category: "Home
Care"
}
]);
```

Output:

```
{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("67f6a57413545e3b0785613b"),
    "1": ObjectId("67f6a57413545e3b0785613c")
  }
}
> db.salesData.find();

[
  {
    "_id": ObjectId("67f6a55013545e3b0785613a"),
    "productName": "Dove Shampoo",
    "productNumber": "PRD1001", "brand":
    "Dove", "category": "Personal Care"
  },
  {
    "_id": ObjectId("67f6a57413545e3b0785613b"),
    "productName": "Maggi Noodles",
    "productNumber": "PRD1002", "brand":
    "Nestle",
    "category": "Food"
```

```
},  
  
{  
  "_id": ObjectId("67f6a57413545e3b0785613c"),  
  "productName": "Surf Excel",  
  "productNumber": "PRD1003", "brand":  
    "HUL",  
  "category": "Home Care"  
}
```

Update UpdateOne

```
db.inventoryData.updateOne({ name: "Dove Shampoo" }, { $set: { stockQuantity: 500 } });

{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedCount": 0
}
```

UpdateMany

Query:

```
db.inventoryData.updateMany({}, { $inc: { stockQuantity: 5 } });
```

Output:

```
>db.inventoryData.find()
```

; Delete

DeleteOne

```
db.inventoryData.deleteOne({ stockId: 2 });
```

```
{
  "acknowledged": true,
  "deletedCount": 1
}

{
  "_id": ObjectId("6776a55013545e3b0785613a"),
  "name": "Dove Shampoo",
  "stockId": 1,
  "category":
    "Personal Care", "stockQuantity":
    550
}

{
  "_id": ObjectId("6776a57413545e3b0785613c"),
```

```
"name": "Maggi  
Noodles",  
"stockId": 3,  
"category": "Food",  
"stockQuantity":  
450  
}
```

DeleteMany:

Query

```
db.inventoryData.deleteMany({ category: "Food" });  
  
{  
  acknowledged: true,  
  deletedCount: 1  
}
```

Output:

Query:

```
{  
  "_id": ObjectId("676a555013545e3b0785613a"),  
  "name": "Dove Shampoo",  
  "stockId": 1,  
  "category":  
    "Personal Care", "stockQuantity":  
    550  
}
```

Create collection “employee”

```
>db.createCollection(“employees”);
```

Create:

InsertOne:

```
db.employees.insertOne({ name: "John", position: "Sales Executive", salary: 40000 });
```

```
{
  "acknowledged": true,
  "insertedId": ObjectId("67f6ac3abd60c89ad017ca54")
}
```

InsertMany

```
db.employees.insertMany([
  { name: "Emma", position: "Regional Manager", salary: 60000 },
  { name: "Liam", position: "Sales Analyst", salary: 45000 }
]);
```

```
{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("G7f6ac76bd60c89ad017ca55"),
    "1": ObjectId("G7f6ac76bd60c89ad017ca56")
  }
}
```

Update

UpdateOne:

Query:

```
db.salesData.updateOne(
  { productName: "Dove Shampoo" },
  { $set: { category: "Beverages" } }
);
```

Output:

```
{
  "acknowledged": true,
  "insertedId": null,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedCount": 0
}
```

UpdateMany:**Query:**

```
db.salesData.updateMany(  
  {},  
  { $set: { salesStatus: "Completed" } }  
);
```

Output:

```
> db.salesData.find();  
  
[  
  {  
    "_id": ObjectId("67f6a55013545e3b0785613a"),  
    "productName": "Dove Shampoo",  
    "productNumber": "PRD1001", "brand":  
    "Dove", "category": "Beverages",  
    "salesStatus": "Completed"  
  },  
  {  
    "_id": ObjectId("67f6a57413545e3b0785613b"),  
    "productName": "Maggi Noodles",  
    "productNumber": "PRD1002", "brand":  
    "Nestle",  
    "category": "Food",  
    "salesStatus": "Completed"  
  },  
  {  
    "_id": ObjectId("67f6a57413545e3b0785613c"),  
    "productName": "Surf Excel",  
    "productNumber": "PRD1003",
```

"brand": "HUL", "category":

"Home Care", "salesStatus":

"Completed"

}

]

Delete

DeleteOne:

Query:

```
db.salesData.deleteOne({ productNumber: "PRD1002" });
```

Output:

{

"acknowledged": true,

"deletedCount": 1

}

{

"_id": ObjectId("67f6a55013545e3b0785613a"),

"productName": "Dove Shampoo",

"productNumber": "PRD1001", "brand":

"Dove", "category": "Beverages",

"salesStatus": "Completed"

}

{

"_id": ObjectId("67f6a57413545e3b0785613c"),

"productName": "Surf Excel",

"productNumber": "PRD1003", "brand":

"HUL",

"category": "Home Care",

"salesStatus": "Completed"

}

DeleteMany:**Query:**

```
db.salesData.deleteMany({ category: "Home Care" });
```

Output:

```
{  
  "acknowledged": true,  
  "deletedCount": 1  
}
```

Output:

```
{  
  "_id": ObjectId("67f6a55013545e3b0785613a"),  
  "productName": "Dove Shampoo",  
  "productNumber": "PRD1001", "brand":  
    "Dove", "category": "Beverages",  
  "salesStatus": "Completed"  
}
```

Result:

Thus, the above queries to perform CRUD operations on various collections is executed successful.

Ex. No. 4a	PROJECTION
Date:	

Aim:

To perform an projection operation on SalesData collection.

PROCEDURE:

Create a collection “SalesData”

```
> db.createCollection("SalesData")
```

Insert the data

```
db.SalesData.insertMany([
{
productName: "Dove Shampoo",
brand: "Dove", category: "Personal
Care",
features: ["GPS", "ABS", "Airbags"],
inventory: [
{ title: "Tea Products", status: "Completed" },
{ title: "Brake Check", status: "In Progress" }
]
},
{
productName: "Maggi
Noodles", brand: "Nestle",
category: "Food",
features: ["Bluetooth", "Reverse Camera", "ABS"], inventory:
[
{ title: "Snacks", status: "Completed" },
{ title: "Battery Check", status: "Pending" }
]
},
{
productName: "Surf Excel",
brand: "HUL",
```

```

category: "Home Care",
features: ["Airbags", "Touchscreen", "GPS"],
inventory: [
  { title: "Oil Service", status: "Completed" },
  { title: "Engine Inspection", status: "Completed" }
]
}
]

```

\$Match

\$match filters documents in the pipeline based on given conditions. It works like the find() method and is used to narrow down data early in the aggregation process.

Query:

```

db.SalesData.aggregate([
  {
    "$match": {
      "category": "Food"
    }
  }
])

```

Output

```

[
  {
    "_id": 2,
    "productName": "Maggi
    Noodles", "brand": "Nestle",
    "category": "Food",
    "features": ["Bluetooth", "Reverse Camera", "ABS"],
    "inventory": [...]
  }
]

```

\$ElemMatch

\$elemMatch is used to match specific elements inside an array field that meet certain criteria. It ensures that all conditions apply to the same array element.

Query:

```

db.SalesData.aggregate([

```

```
{
"$match": {
"inventory": {
"$elemMatch": {
"status": "Pending"
}
}
}
}
})
```

Output

```
[
{
"_id": 2,
"productName": "Maggi Noodles",
"inventory": [
{ "title": "Snacks", "status": "Completed" },
{ "title": "Battery Check", "status": "Pending" }
]
}
]
```

\$Slice

\$slice returns a subset of array elements. You can specify how many items to return from the start or use it like pagination.

Query

```
db.SalesData.aggregate([
{
"$project": {
"productName":
1, "features": {
"$slice": ["$features", 2]
}
}
}
})
```

Output

```
[
  {
    "productName": "Dove Shampoo",
    "features": ["GPS", "ABS"]
  },
  {
    "productName": "Maggi Noodles",
    "features": ["Bluetooth", "Reverse Camera"]
  },
  {
    "productName": "Surf Excel",
    "features": ["Airbags",
    "Touchscreen"]
  }
]
```

\$Meta

\$meta is used to return metadata about documents — typically used with text search to get the relevance score of how well a document matches a search term.

Make sure the Text Index exists

```
db.SalesData.createIndex({ features: "text"
})
```

Query

```
db.SalesData.aggregate([
  {
    "$match": {
      "$text": {
        "$search": "GPS"
      }
    }
  },
  {
    "$project": {
      "productName":
1,
```

"features": 1,

"score": {

"\$meta": "textScore"

```
}  
}  
},  
{  
"$sort": {  
  "score": {  
    "$meta": "textScore"  
  }  
}  
}  
})
```

Output:

```
[  
  {  
    "productName": "Dove Shampoo",  
    "features": ["GPS", "ABS", "Airbags"],  
    "score": 1.5  
  },  
  {  
    "productName": "Surf Excel",  
    "features": ["Airbags", "Touchscreen", "GPS"],  
    "score": 1.5  
  }  
]
```

Result:

Thus, the queries to perform various Projection operations on SalesData collection is executed successfully.

Ex. No. 4b	AGGREGATION
Date:	

Aim:

To perform an aggregate operation on Sales collection.

PROCEDURE:

Insert

```
db.Inventory.insertMany([
{
productName: "Dove
Shampoo", category: "Personal
Care", cost: 3000,
salesCount: 2,
serviceCenter: "AutoCare Center"
},
{
productName:      "Maggi
Noodles", category: "Food",
cost: 5000,
salesCount: 1, serviceCenter:
"Speed Motors"
},
{
productName: "Surf Excel",
category: "Home Care", cost:
7000,
salesCount: 3,
serviceCenter: "AutoCare Center"
},
{
productName: "Parle-G",
```



```

category: "Snacks", cost: 4000,
salesCount: 4, serviceCenter:
"Wheel Zone"
},
{
productName: "Aashirvaad
Atta", category: "Groceries",
cost: 6000,
salesCount: 1, serviceCenter:
"CarFix Hub"
},
{
productName: "Coca Cola", category:
"Beverages",
cost: 50000,
salesCount: 2,
serviceCenter: "AutoCare Center"
}
])

```

1. \$match

```

db.Inventory.aggregate([
{
$match: {
category: "Personal Care"
}
}
])

```

Output

```

[
{
productName: "Dove Shampoo",

```

```
category: "Personal Care",
cost: 3000,
salesCount: 2,
serviceCenter: "AutoCare Center"
}
]
```

2. \$project

```
db.Inventory.aggregate([
{
  $project: {
    productName:
    1,
    cost: 1,
    salesCount: 1,
    totalInventoryCost: {
      $multiply: ["$cost", "$salesCount"]
    }
  }
}
])
```

Output

```
[
{
  productName: "Dove
Shampoo", cost: 3000,
salesCount: 2,
totalInventoryCost: 6000
},
{
```

productName: "Maggi Noodles",

```

cost: 5000,

salesCount: 1,

totalInventoryCost: 5000

}

]

```

3. \$group

```

db.Inventory.aggregate([
{
  $group: {
    _id: "$serviceCenter",
    totalServices: { $sum: "$salesCount" },
    averageCost: { $avg: "$cost" }
  }
}
])

```

Output

```

[
{
  _id: "AutoCare Center",
  totalServices: 7,
  averageCost: 6333.33
},
{
  _id: "Speed Motors",
  totalServices: 1,
  averageCost: 5000
},
{
  _id: "Wheel Zone",
  totalServices: 4,
  averageCost: 4000
}
]

```

4. \$sum

```

db.Inventory.aggregate([

{

  $group: {

    _id: "$category",

    totalServices: { $sum: "$salesCount" }

  }

}

])

```

Output

```
[
  {
    _id: "Personal Care",
    totalServices: 2
  },
  {
    _id: "Food",
    totalServices: 1
  },
  {
    _id: "Home Care",
    totalServices: 3
  },
  {
    _id: "Snacks",
    totalServices: 4
  }
]
```

5. \$avg

```
db.Inventory.aggregate([
  {
    $group: {
      _id: "$category", avgCost:
      { $avg: "$cost" }
    }
  }
])
```

Output

```
[
  {
    _id: "Personal Care",
```

```

    avgCost: 3000
  },
  {
    _id: "Food", avgCost: 5000
  },
  {
    _id: "Home Care", avgCost:
7000
  },
  {
    _id: "Snacks", avgCost:
4000
  }
]
]

```

6. \$min

```

db.Inventory.aggregate([
  {
    $group: {
      _id: "$category", minCost: {
        $min: "$cost" }
    }
  }
])

```

Output

```

[
  {
    _id: "Personal Care",
    minCost: 3000
  },
  {
    _id: "Food",
    minCost: 5000
  },
  {
    _id: "Home Care",
    minCost: 7000
  },
  {
    _id: "Snacks", minCost:
4000
  }
]

```

]

7. \$max

]

```
db.Inventory.aggregate([
  {
    $group: {
      _id: "$category", maxCost:
      { $max: "$cost" }
    }
  }
])
```

Output

```
[
  {
    _id: "Personal
    Care", maxCost:
    3000
  },
  {
    _id: "Food", maxCost: 5000
  },
  {
    _id: "Home Care", maxCost:
    7000
  },
  {
    _id: "Snacks", maxCost:
    4000
  }
]
```

8. \$sort

```
db.Inventory.aggregate([
  {
    $sort: { cost: -1 }
  }
])
```

Output

```
[
  {
    productName: "Coca Cola", cost:
    50000,
    category: "Beverages"
  },
  {
    productName: "Surf Excel",
    cost: 7000,
    category: "Home Care"
  },
  {

```

```
productName: "Aashirvaad  
Atta", cost: 6000,  
category: "Groceries"  
}]
```

Result:

Thus, the query to perform various Aggregation operations on Inventory collection are executed successfully.

Ex. No. 5	DATA MODELING
Date:	

Aim:

To implement embedded and reference model example using product inventory collection.

Embedded Model

Procedure:

```
db.SalesInventory.insertMany([
```

```
{
```

```
  _id: 1,
```

```
  productName: "Dove
```

```
  Shampoo",
```

```
  category: "Personal
```

```
  Care", salesCount: 2,
```

```
  inventoryHistory: [
```

```
    { date: "2024-03-01",
```

```
    cost: 3000, status:
```

```
    "Completed" },
```

```
    { date: "2024-03-05",
```

```
    cost: 1500, status:
```

```
    "Pending" }
  ]
},
```

```
{
```

```
{
```

```
  _id: 2,
```

```
  _id: 2,
```

```
  productName: "Maggi
```

```
  Noodles",
```

```
  category: "Food",
```

```
salesCount: 3,
inventoryHistory: [
  { date: "2024-03-02",
cost: 5000, status:
"Completed" },
  { date: "2024-03-07",
cost: 2000, status: "In
Progress" },
  { date: "2024-03-10",
cost: 1000, status:
"Pending" }
],
},
{
  _id: 3,
productName: "Surf
Excel",
category: "Home
Care", salesCount: 1,
inventoryHistory: [
  { date: "2024-03-01",
cost: 7000, status:
"Completed" },
  { date: "2024-03-06",
cost: 2500, status:
"Pending" }
],
}
])
```

Get latest inventory record for each product:

```
db.SalesInventory.aggregate([  
  {  
    $project: {  
      productName: 1,  
      salesCount: 1,  
      lastInventory: { $arrayElemAt: ["$inventoryHistory", -1] }  
    }  
  }  
])
```

Output

```
[  
  {  
    productName: "Dove Shampoo",  
    salesCount: 2, lastInventory: {  
      date: "2024-03-05",  
      cost: 1500,  
      status: "Pending"  
    }  
  },  
  {  
    productName: "Maggi Noodles",  
    salesCount: 3, lastInventory: {  
      date: "2024-03-10",  
      cost: 1000,  
      status: "Pending"  
    }  
  },  
  {  
    productName: "Surf Excel",  
    salesCount: 1,
```

```
lastInventory: {  
  date: "2024-03-06",  
  cost: 2500,  
  status: "Pending"  
}  
}  
]
```

Count number of inventory records per product:

```
db.SalesInventory.aggregate([  
  {  
    $project: {  
      productName:  
        1,  
      inventoryCount: { $size: "$inventoryHistory" }  
    }  
  }  
])
```

Output:

```
[  
  { productName: "Dove Shampoo", inventoryCount: 2 },  
  { productName: "Maggi Noodles", inventoryCount: 3 },  
  { productName: "Surf Excel", inventoryCount: 2 }  
]
```

Reference Data Model

Procedure:

```
db.SalesInventory.insertMany([  
  { _id: 1, productName: "Dove Shampoo", category: "Personal Care", salesCount: 2 },  
  { _id: 2, productName: "Maggi Noodles", category: "Food", salesCount: 3 },  
  { _id: 3, productName: "Surf Excel", category: "Home Care", salesCount: 1 }  
])
```

```
db.InventoryHistory.insertMany([  
  { _id: 101, productId: 1, date: "2024-03-01", cost: 3000, status: "Completed" },  
  { _id: 102, productId: 1, date: "2024-03-05", cost: 1500, status: "Pending" },
```

```
{ _id: 103, productId: 2, date: "2024-03-02", cost: 5000, status: "Completed" },
{ _id: 104, productId: 2, date: "2024-03-07", cost: 2000, status: "In Progress" },
{ _id: 105, productId: 2, date: "2024-03-10", cost: 1000, status: "Pending" },
{ _id: 106, productId: 3, date: "2024-03-01", cost: 7000, status: "Completed" },
{ _id: 107, productId: 3, date: "2024-03-06", cost: 2500, status: "Pending" }
])
```

Join SalesInventory with InventoryHistory:

```
db.SalesInventory.aggregate([
  {
    $lookup: {
      from: "InventoryHistory",
      localField: "_id",
      foreignField: "productId",
      as: "inventoryHistory"
    }
  }
])
```

Output:

```
{
  _id: 1,
  productName: "Dove Shampoo",
  salesCount: 2, inventoryHistory: [
    { productId: 1, date: "2024-03-01", cost: 3000, status: "Completed" },
    { productId: 1, date: "2024-03-05", cost: 1500, status: "Pending" }
  ]
}
```

Inventory records in March 2024:

```
db.InventoryHistory.find({
```

```
date: { $gte: "2024-03-01", $lte: "2024-03-31" }
}))
```

Output:

```
[
  { productId: 1, date: "2024-03-01", cost: 3000, status: "Completed" },
  { productId: 1, date: "2024-03-05", cost: 1500, status: "Pending" }
]
```

Total inventory cost per product:

```
db.InventoryHistory.aggregate([
  { $match: { cost: { $gt: 0 } } },
  {
    $group: {
      _id: "$productId",
      totalCost: { $sum: "$cost" }
    }
  }
])
```

Output:

```
[
  { _id: 1, totalCost: 4500 },
  { _id: 2, totalCost: 8000 },
  { _id: 3, totalCost: 55000 }
]
```

List salesData with pending inventory and their names:

```
db.InventoryHistory.aggregate([
  { $match: { status: "Pending" } },
  {
    $lookup: {
```

```
from: "SalesInventory",
localField: "productId",
foreignField: "_id",
as: "productInfo"
}
},
{ $unwind: "$productInfo" },
{
$project: {
productName:
"$productInfo.productName", cost: 1,
status: 1,
date: 1
}
}
])
```

Output:

```
[
{ productName: "Dove Shampoo", cost: 1500, status: "Pending", date: "2024-03-05" },
{ productName: "Maggi Noodles", cost: 1000, status: "Pending", date: "2024-03-10" },
{ productName: "Surf Excel", cost: 2500, status: "Pending", date: "2024-03-06" }
]
```

Result:

Thus, the queries to implement data modeling on product inventory collection is executed successfully.

Ex. No. 6	PERFORM SCHEMA DESIGN AND INDEXING
Date:	

Aim

To perform Schema Design and Indexing in MongoDB.

Schema Design:

Let's assume a Digital Sales Inventory & Tracking Database. We'll create a "salesData" collection with fields like:

- _id
- productName
- brand
- category
- salesCount
- inventoryStatus

Data Insertion:

```
db.salesData.insertMany([
```

```
{
  _id: 1,
  productName: "Dove Shampoo",
  brand: "Dove", category: "Personal
  Care", salesAmount: 3000,
  inventoryStatus: "Completed"
},
```

```
{
  _id: 2,
  productName: "Maggi
  Noodles", brand: "Nestle",
  category: "Food",
  salesAmount: 5000,
  inventoryStatus: "Pending"
},
```

```
{
  _id: 3,
  productName: "Surf Excel",
  brand: "HUL", category: "Home
  Care", salesAmount: 7000,
  inventoryStatus: "Completed"
},
```

```
{
  _id: 4,
  productName: "Aashirvaad Atta",
  brand: "ITC", category:
  "Groceries", salesAmount: 4000,
  inventoryStatus: "In Progress"
},
```

```
{
  _id: 5,
  productName: "Coca Cola", brand:
  "Coca Cola",
```



```
category: "Beverages",  
salesAmount: 50000,  
inventoryStatus: "Completed"  
}
```

```
]);
```

Indexing:

1. Create index on the category field to speed up service-based

queries: Query

```
> db.salesData.createIndex({ category: 1 });
```

2. Create compound index

salesAmount and inventoryStatus:

: Query

```
> db.salesData.createIndex({ salesAmount: 1, inventoryStatus: -1 });
```

Queries:

Find all salesData with "Personal Care" service type:

```
> db.salesData.find({ category: "Personal Care" });
```

Output:

```
[  
  
  {  
  
    "_id": 1,  
  
    "productName": "Dove Shampoo",  
  
    "brand": "Dove", "category": "Personal  
Care", "salesAmount": 3000,  
  
    "inventoryStatus": "Completed"  
  }  
]
```

Find salesData with service cost less than 5000:

```
> db.salesData.find({ salesAmount: { $lt: 5000 } });
```

Output:

```
[
  {
    "_id": 1,
    "productName": "Dove Shampoo",
    "brand": "Dove", "category": "Personal
    Care", "salesAmount": 3000,
    "inventoryStatus": "Completed"
  },
  {
    "_id": 4,
    "productName": "Aashirvaad Atta",
    "brand": "ITC", "category":
    "Groceries", "salesAmount": 4000,
    "inventoryStatus": "In Progress"
  }
]
```

Check all created indexes:

```
> db.salesData.getIndexes();
```

Output:

```
[
  {
    "v": 2,
    "key": { "_id": 1 },
    "name": "_id_"
  },
  {
    "v": 2,
    "key": { "category": 1 }, "name":
    "category_1"
  },
]
```

```
{  
  "v": 2,  
  "key": { "salesAmount": 1, "inventoryStatus": -1 },  
  "name": "salesAmount_1_inventoryStatus_-1"  
}  
]
```

Result:

Thus, the schema design and indexing operations on product inventory collection in MongoDB were successfully performed.

Ex. No. 7	BUILDING AGGREGATION PIPELINES
Date:	

Aim:

To perform aggregation pipeline operations for SalesData and Inventory collections in MongoDB.

Sales Collection:

```
db.salesData.insertMany([
    { productId: 1, productName: "Dove Shampoo", brand: "Nestle", category: "Personal Care",
      salesAmount: 3000 },
    { productId: 2, productName: "Maggi Noodles", brand: "HUL", category: "Food",
      salesAmount: 5000 },
    { productId: 3, productName: "Surf Excel", brand: "ITC", category: "Home Care", salesAmount:
      7000 },
    { productId: 4, productName: "Aashirvaad Atta", brand: "Coca Cola", category: "Groceries",
      salesAmount: 4000 },
    { productId: 5, productName: "Coca Cola", brand: "Amul", category: "Beverages",
      salesAmount: 50000 }
])
```

Aggregation pipeline queries:

Filter salesData with service cost greater than 4000

```
>db.salesData.aggregate([
    { $match: { salesAmount: { $gt: 4000 } } }
])
```

Output:

```
{
  productId: 2,
  productName: "Maggi
Noodles", brand: "HUL",
  category: "Food",
  salesAmount: 5000
}
{
  productId: 3,
```

```
productName: "Surf Excel",  
brand: "ITC", category: "Home  
Care", salesAmount: 7000  
}
```

```
{  
productId: 5,  
productName: "Coca Cola", brand: "Amul",  
category: "Beverages",  
salesAmount: 50000  
}
```

Group salesData by service type and get average service cost

```
>db.salesData.aggregate([  
  
{  
$group: {  
_id: "$category",  
avgServiceCost: { $avg: "$salesAmount" },  
count: { $sum: 1 }  
}  
}  
])
```

Output:

```
{  
_id: "Personal Care",  
avgServiceCost: 3000,  
count: 1  
}  
  
{  
_id: "Food",  
avgServiceCost: 5000,  
count: 1
```

```

}

{
  _id: "Home Care",
  avgServiceCost: 7000,
  count: 1
}

```

Project product name and a new field 'serviceLevel'

```

>db.salesData.aggregate([
  {
    $project: {
      productName:
      1,
      salesAmount: 1,
      serviceLevel: {
        $switch: {
          branches: [
            { case: { $gte: ["$salesAmount", 8000] }, then: "Premium" },
            { case: { $gte: ["$salesAmount", 5000] }, then: "Standard" },
            { case: { $gte: ["$salesAmount", 3000] }, then: "Basic" }
          ],
          default: "Low"
        }
      }
    }
  }
])

```

Output:

```

{
  productName: "Dove
Shampoo", salesAmount: 3000,
  serviceLevel: "Basic"
}
{

```

productName: "Maggi Noodles",

```
salesAmount: 5000,  
  
serviceLevel: "Standard"  
  
}  
  
{  
  
productName: "Coca Cola", salesAmount:  
50000,  
  
serviceLevel: "Premium"  
  
}
```

Sort salesData by service cost descending

```
>db.salesData.aggregate([  
  
{ $sort: { salesAmount: -1 } }  
  
])
```

Output:

```
{  
  
productId: 5,  
  
productName: "Coca Cola", brand: "Amul",  
  
category: "Beverages",  
  
salesAmount: 50000  
  
}  
  
{  
  
productId: 3,  
  
productName: "Surf Excel",  
  
brand: "ITC", category: "Home  
Care", salesAmount: 7000  
  
}  
  
{  
  
productId: 2,  
  
productName: "Maggi  
Noodles", brand: "HUL",  
  
category: "Food",  
  
salesAmount: 5000}
```


Result:

Thus the above queries to perform aggregation pipeline operations for SalesData and Inventory collections in MongoDB are executed successfully.

Ex. No. 8	PERFORMING DATA TRANSFORMATION
Date:	

Aim:

To perform a Data Transformation operation on Sales Inventory Management System.

Sales Inventory collection:

```
db.inventory.insertMany([
  { productName: "Dove Shampoo", category: "Personal Care", salesCount: 2, salesAmount: 3000 },
  { productName: "Maggi Noodles", category: "Food", salesCount: 5, salesAmount: 5000 },
  { productName: "Surf Excel", category: "Home Care", salesCount: 3, salesAmount: 7000 },
  { productName: "Aashirvaad Atta", category: "Groceries", salesCount: 4, salesAmount: 4000 },
  { productName: "Coca Cola", category: "Beverages", salesCount: 1, salesAmount: 50000 },
  { productName: "Parle-G", category: "Snacks", salesCount: 6, salesAmount: 2000 },
  { productName: "Clinic Plus", category: "Household Products", salesCount: 7, salesAmount: 3500 },
  { productName: "Sunfeast Biscuits", category: "Hair Care", salesCount: 8, salesAmount: 4500 },
  { productName: "Amul Butter", category: "Dairy", salesCount: 2, salesAmount: 15000 },
  { productName: "Tata Tea", category: "Tea Products", salesCount: 1, salesAmount: 18000 }
])
```

Bucket (group salesData based on service cost):

```
db.inventory.aggregate([
  {
    $bucket: {
      groupBy: "$salesAmount",
      boundaries: [0, 3000, 6000, 12000, 20000],
      default: "Other",
      output: {
        count: { $sum: 1 },
        salesData: { $push: "$productName" }
      }
    }
  }
])
```

```
}
```

```
}
```

```
])
```

Output:

```
{
```

```
  _id: 0,
```

```
  count: 1,
```

```
  salesData: [
```

```
    'Parle-G'
```

```
  ]
```

```
}
```

```
{
```

```
  _id: 3000,
```

```
  count: 5,
```

```
  salesData: [
```

```
    'Dove
```

```
    Shampoo',
```

```
    'Maggi
```

```
    Noodles',
```

```
    'Aashirvaad
```

```
    Atta', 'Clinic
```

```
    Plus', 'Sunfeast
```

```
    Biscuits'
```

```
  ]
```

```
}
```

```
{
```

```
  _id: 6000,
```

```
  count: 2,
```

```
  salesData: [
```

```
    'Surf Excel',
```

```
    'Coca Cola'
```

```
  ]
```

```
}
```

{

_id: 12000,

count: 2,

salesData:

['Amul

Butter',

'Tata Tea'

]

}

Facet (multiple aggregations in one):

db.inventory.aggregate([

{

\$facet: {

"ByServiceType": [

{

\$group: {

_id: "\$category",

totalSalesData: { \$sum: 1 }

}

}

],

"HighServiceCount": [

{

\$match: {

salesCount: { \$gte: 5 }

}

},

{

\$project: {

productName: 1,

salesCount: 1,

_id: 0

}

}

]

```
}
```

```
}
```

```
])
```

Output:

```
{
```

```
  ByServiceType: [
```

```
    {
```

```
      _id: 'Personal Care',
```

```
      totalSalesData: 1
```

```
    },
```

```
    {
```

```
      _id: 'Food',
```

```
      totalSalesData: 1
```

```
    },
```

```
    {
```

```
      _id: 'Home Care',
```

```
      totalSalesData: 1
```

```
    },
```

```
    {
```

```
      _id: 'Groceries', totalSalesData:
```

```
      1
```

```
    }  
  ],
```

```
  HighServiceCount: [
```

```
    {
```

```
      productName: 'Maggi Noodles',
```

```
      salesCount: 5
```

```
    },
```

```
    {
```

```
      productName: 'Parle-G',
```

```
      salesCount: 6
```

```
},  
  
{  
  
  productName: 'Clinic Plus',  
  
  salesCount: 7  
  
},  
  
{  
  
  productName: 'Sunfeast  
Biscuits', salesCount: 8  
  
}  
  
]  
  
}
```

Sort by service cost (high to low):

db.inventory.find().sort({ salesAmount: -1 })

Output:

```
{  
  
  productName: 'Tata Tea',  
  
  category: 'Tea Products',  
  
  salesCount: 1,  
  
  salesAmount: 18000  
  
}  
  
{  
  
  productName: 'Amul Butter',  
  
  category: 'Dairy', salesCount: 2,  
  
  salesAmount: 15000  
  
}  
  
{  
  
  productName: 'Coca Cola',  
  
  category: 'Beverages', salesCount:  
  
  1,  
  
  salesAmount: 50000  
  
}
```

```
{
  productName: 'Surf Excel',
  category: 'Home Care',
  salesCount: 3,
  salesAmount: 7000
}
{
  productName: 'Maggi
  Noodles', category: 'Food',
  salesCount: 5,
  salesAmount: 5000
}
```

Skip first 2 documents:

db.inventory.find().skip(2)

Output:

```
{
  productName: 'Surf Excel',
  category: 'Home Care',
  salesCount: 3,
  salesAmount: 7000
}
{
  productName: 'Aashirvaad
  Atta', category: 'Groceries',
  salesCount: 4,
  salesAmount: 4000
}
{
  productName: 'Coca Cola',
  category: 'Beverages', salesCount:
  1,
```


salesAmount: 50000

}

{

productName: 'Parle-G', category:

'Snacks', salesCount: 6,

salesAmount: 2000

}

Limit to 5 documents:

db.inventory.find().limit(5)

Output:

{

productName: 'Dove Shampoo',

category: 'Personal Care',

salesCount: 2,

salesAmount: 3000

}

{

productName: 'Maggi

Noodles', category: 'Food',

salesCount: 5,

salesAmount: 5000

}

{

productName: 'Surf Excel',

category: 'Home Care',

salesCount: 3,

salesAmount: 7000

}

{

productName: 'Aashirvaad

Atta', category: 'Groceries',

```
salesCount: 4,  
salesAmount: 4000  
}  
{  
productName: 'Coca Cola',  
category: 'Beverages', salesCount:  
1,  
salesAmount: 50000  
}
```

Result:

Thus, the above queries to perform Data Transformation on the 'Sales Inventory' collection in MongoDB have been executed successfully.

Ex. No. 9	ESTABLISHING CONNECTION BETWEEN MONGODB (COMPASS) AND JAVA(ECLIPSE)
Date:	

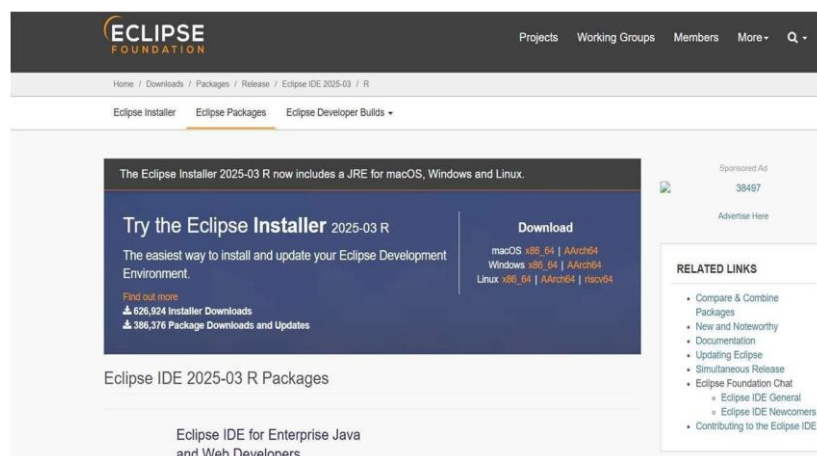
AIM:

To establish a connection between a Java program and a MongoDB database using Maven dependencies, insert a document into the database, and verify the insertion through MongoDB Compass.

PROCEDURE:

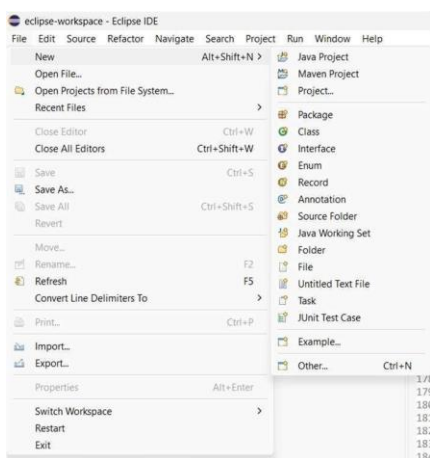
Step 1: Set Up the Environment

- Install **Eclipse IDE** from <https://eclipseide.org/> (if not already installed).



Step 2: Create a New Java Project

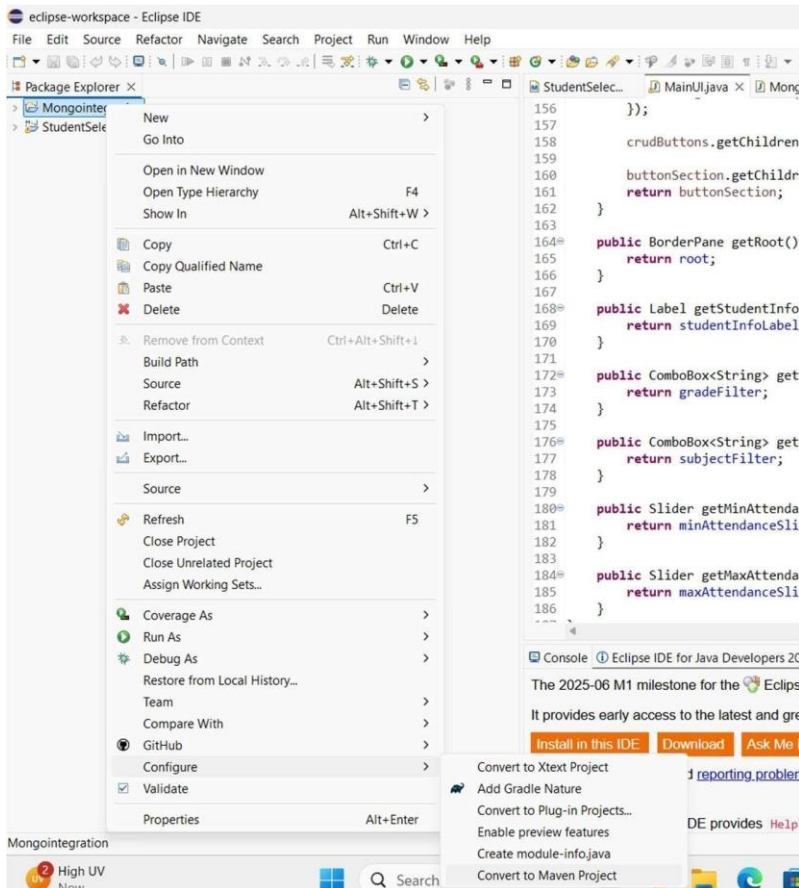
- Open Eclipse.
- Go to File > New > Java Project.



- Name the project (e.g., MongoIntegration).

Step 3: Convert to Maven Project

- Right-click the project > Configure > Convert to Maven Project.



Step 4: Add Maven Dependency

- Open the pom.xml file.
- Add the following dependency inside the <dependencies> tag:

```
<dependency>
<groupId>org.mongodb</groupId>
<artifactId>mongodb-driver-sync</artifactId>
<version>4.11.0</version>
</dependency>
```

```

1 https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://m
3 <groupId>MongoIntegration</groupId>
4 <artifactId>MongoIntegration</artifactId>
5 <version>0.0.1-SNAPSHOT</version>
6 <build>
7 <sourceDirectory>src</sourceDirectory>
8 <plugins>
9 <plugin>
10 <artifactId>maven-compiler-plugin</artifactId>
11 <version>3.13.0</version>
12 <configuration>
13 <release>21</release>
14 </configuration>
15 </plugin>
16 </plugins>
17 </build>
18 </project>

```

- Save the pom.xml. Maven will automatically download the MongoDB driver library.

Step 5: Write Java Code

- Create a Java class (e.g., MongoConnect.java).
- Write the following code to connect to MongoDB, insert data, and retrieve documents:

```

import
com.mongodb.client.*;
import
org.bson.Document;

public class MongoConnect {
    public static void main(String[] args)
    { String uri =
      "mongodb://localhost:27017";
      MongoClient mongoClient = MongoClient.create(uri);

      MongoDB database = mongoClient.getDatabase("testDB");
      MongoCollection<Document> collection =
        database.getCollection("inventoryData");

      Document doc = new Document("name", "Sahana")
        .append("age", 21)
        .append("department", "IT");
      collection.insertOne(doc);

      FindIterable<Document> docs =
        collection.find(); for (Document d : docs) {
        System.out.println(d.toJson());
      }

      mongoClient.close();
    }
}

```

Step 6: Run the Program

- Right-click the file > Run As > Java Application.
- Observe the console output displaying inserted documents.

Step 7: Verify in MongoDB Compass

- Open MongoDB Compass.
- Connect to mongodb://localhost:27017.
- Navigate to testDB > inventoryData collection.
- Verify that the document (e.g., Sahana, 21, IT) is inserted.

Result:

Thus, the connection between MongoDB (compass) and JAVA(eclipse) is established successfully.

Ex. No. 10	Develop a CRUD Application using MongoDB(Mini Project)
Date:	

Aim:

To develop an FMCG Analytics and Prediction web application using MongoDB as the database to store product, sales, and prediction data, enabling efficient data management and real-time insights.

Procedure:

Step:1 – Design and develop the frontend using Streamlit for an interactive dashboard interface.

Step:2 – Develop the backend using FastAPI (Python) for handling APIs and business logic.

Step:3 – Establish connection between FastAPI and MongoDB using PyMongo/Motor.

Step:4 – Create backend API routes to send and receive data between the database and server.

Step:5 – Integrate the backend APIs with the Streamlit frontend or real-time data display and predictions.

Step:6 – Integrate frontend with backend and test the application.

Code:

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
import plotly.express as px
import plotly.graph_objects as go
from datetime import datetime
import importlib
import database
importlib.reload(database)

import database as db

# _____
# Page config (must be first Streamlit call)
# _____
st.set_page_config(
    page_title="FMCG Nexus",
    page_icon="💎",
    layout="wide",
    initial_sidebar_state="expanded",
)

# _____
# GLOBAL STYLES
#
# KEY DECISIONS that fix the four bugs:
#
# BUG 1 & 2 (toolbar text / sidebar text)
```

```

# The original code used * { font-family: Sora }
# That wildcard overrides Streamlit's internal Material
# Icons font, so icon ligatures like "contrast_mode" and
# "keyboard_double_arrow_right" render as raw text.
# FIX: scope the Sora font override ONLY to html/body and
# specific UI elements. Never use the wildcard *.
#
# BUG 3 (light mode — only toolbar changes)
# Streamlit's theme toggle overrides background on every
# layout wrapper div. Our dark colour was only on .stApp.
# FIX: lock background-color on EVERY Streamlit wrapper
# selector so the toggle cannot bleed through anywhere.
#
# BUG 4 (blank table + jumbled column menu + ghost glass box)
# Three sub-causes:
# a) [data-testid="stTabsContent"] was included in the glass
# card CSS rule → whole tab panel got a glass box, creating
# the unwanted floating card above the metrics.
# b) .glass-section had `overflow: hidden` which clips the
# GlideDataEditor canvas that renders table rows.
# c) * font override breaks Material Icons inside the
# dataframe column-header context menu.
# FIX: remove stTabsContent from glass rule, remove
# overflow:hidden from .glass-section, keep font scoped.
# _____
st.markdown(
    """
<style>
/* — Google Font — */
@import
url('https://fonts.googleapis.com/css2?family=Sora:wght@200;300;400;500;600;700;800&display=swap');

/* _____
VARIABLES
_____ */

:root {
  --bg: #020408;
  --glass-bg: rgba(255,255,255,0.07);
  --glass-bdr: rgba(255,255,255,0.16);
  --glass-shd: 0 8px 32px rgba(0,0,0,0.37);
  --blur: blur(18px);
  --blue: #3b82f6;
  --purple: #8b5cf6;
  --cyan: #06b6d4;
  --green: #10b981;
  --pink: #ec4899;
  --amber: #f59e0b;
  --red: #ef4444;
  --orange: #f97316;
  --t1: rgba(255,255,255,0.95);
  --t2: rgba(255,255,255,0.60);
  --t3: rgba(255,255,255,0.35);
  --r: 20px;
  --rs: 12px;

```



```
}
```

```
/* =====  
BUG FIX 1 and 2  
Scope font to specific elements only.  
NEVER use * { font-family } — that breaks  
Material Icons used by Streamlit's toolbar  
and sidebar-collapse button, causing them  
to show raw ligature text instead of icons.  
===== */
```

```
html, body,  
.stApp,  
p, h1, h2, h3, h4, h5, h6,  
label, div.stMarkdown,  
.stButton > button,  
.stTextInput input,  
.stNumberInput input,  
.stTextArea textarea,  
.stSelectbox > div,  
.stRadio label,  
.stTabs [data-baseweb="tab"],  
[data-testid="stMetricLabel"],  
[data-testid="stMetricValue"],  
[data-testid="stMetricDelta"],  
.glass-section,  
.stat-card,  
.page-chip,  
.sidebar-brand {  
  font-family: 'Sora', sans-serif !important;  
  -webkit-font-smoothing: antialiased;  
}
```

```
/* =====  
BUG FIX 3  
Lock EVERY Streamlit layout wrapper to our  
dark background so the theme toggle cannot  
override any of them.  
===== */
```

```
html,  
body,  
.stApp,  
[data-testid="stAppViewContainer"],  
[data-testid="stMain"],  
[data-testid="block-container"],  
section.main,  
div.main,  
.block-container {  
  background-color: var(--bg) !important;  
  color: var(--t2) !important;  
}
```

```
header[data-testid="stHeader"] {  
  background: rgba(2,4,8,0.90) !important;  
  backdrop-filter: blur(12px) !important;
```

```

border-bottom: 1px solid rgba(255,255,255,0.06) !important;
}

/* — Mesh animated background — */
.stApp {
background: var(--bg) !important;
min-height: 100vh;
position: relative;
overflow-x: hidden;
}
.stApp::before {
content: "";
position: fixed;
inset: 0;
background:
radial-gradient(ellipse 80% 60% at 20% 10%, rgba(59,130,246,.25) 0%, transparent 60%),
radial-gradient(ellipse 60% 80% at 80% 20%, rgba(139,92,246,.20) 0%, transparent 60%),
radial-gradient(ellipse 70% 50% at 50% 80%, rgba(6,182,212,.15) 0%, transparent 60%),
radial-gradient(ellipse 40% 40% at 10% 90%, rgba(236,72,153,.12) 0%, transparent 50%);
animation: meshFlow 20s ease-in-out infinite alternate;
pointer-events: none;
z-index: 0;
}
.stApp::after {
content: "";
position: fixed;
inset: 0;
background-image: radial-gradient(circle at 1px 1px, rgba(255,255,255,.04) 1px, transparent 0);
background-size: 40px 40px;
pointer-events: none;
z-index: 0;
}
@keyframes meshFlow {
0% { opacity:1; transform: scale(1) rotate(0deg); }
33% { opacity:.85; transform: scale(1.05) rotate(1deg); }
66% { opacity:.95; transform: scale(.97) rotate(-1deg); }
100% { opacity:1; transform: scale(1.02) rotate(.5deg); }
}

/* — Floating orb keyframes — */
@keyframes orb1 {
0% { transform: translate(0,0) scale(1); }
25% { transform: translate(80px,-120px) scale(1.15); }
50% { transform: translate(160px,40px) scale(.90); }
75% { transform: translate(40px,120px) scale(1.10); }
100% { transform: translate(0,0) scale(1); }
}
@keyframes orb2 {
0% { transform: translate(0,0) scale(1); }
33% { transform: translate(-100px,80px) scale(1.20); }
66% { transform: translate(60px,-80px) scale(.85); }
100% { transform: translate(0,0) scale(1); }
}
@keyframes orb3 {

```

```

0% { transform: translate(0,0) scale(1); }
40% { transform: translate(120px,100px) scale(1.10); }
80% { transform: translate(-60px,-60px) scale(.92); }
100% { transform: translate(0,0) scale(1); }
}

```

```

/* — Page reveal — */
.main .block-container {
padding: 2rem 2.5rem !important;
max-width: 1400px !important;
position: relative;
z-index: 1;
animation: reveal .6s cubic-bezier(.16,1,.3,1) both;
}

```

```

@keyframes reveal {
from { opacity:0; transform: translateY(24px); }
to { opacity:1; transform: translateY(0); }
}

```

```

/* =====
SIDEBAR
===== */

```

```

section[data-testid="stSidebar"] {
background: rgba(8,8,20,.90) !important;
backdrop-filter: blur(40px) saturate(180%) !important;
-webkit-backdrop-filter: blur(40px) saturate(180%) !important;
border-right: 1px solid rgba(255,255,255,.08) !important;
box-shadow: 4px 0 40px rgba(0,0,0,.5) !important;
}
section[data-testid="stSidebar"] > div {
background: transparent !important;
}

```

```

/* =====
TYPOGRAPHY
===== */

```

```

h1 { font-size:2.4rem !important; font-weight:700 !important; color:var(--t1) !important; letter-spacing:-
.03em !important; line-height:1.15 !important; }
h2 { font-size:1.6rem !important; font-weight:600 !important; color:var(--t1) !important; letter-spacing:-
.02em !important; }
h3 { font-size:1.15rem !important; font-weight:600 !important; color:var(--t2) !important; letter-spacing:-
.01em !important; }
p, label { color: var(--t2) !important; }

```

```

/* =====
BUG FIX 4a
GLASS CARD — applied to stMetric and stForm
ONLY. stTabsContent is deliberately excluded.
The old code included stTabsContent here which
caused a floating glass box to appear above the
metrics/dataframe on the Products page.
===== */

```

```

[data-testid="stMetric"],
[data-testid="stForm"] {

```

```

background: var(--glass-bg) !important;
backdrop-filter: var(--blur) !important;
-webkit-backdrop-filter: var(--blur) !important;
border: 1px solid var(--glass-bdr) !important;
border-radius: var(--r) !important;
box-shadow: var(--glass-shd), inset 0 1px 0 rgba(255,255,255,.12) !important;
padding: 1.5rem !important;
transition: all .3s cubic-bezier(.16,1,.3,1) !important;
}
[data-testid="stMetric"]:hover {
  transform: translateY(-3px) !important;
  border-color: rgba(255,255,255,.25) !important;
  box-shadow: var(--glass-shd), 0 20px 60px rgba(59,130,246,.2), inset 0 1px 0 rgba(255,255,255,.18) !important;
}

/* Explicitly reset tab panel — no glass background */
.stTabs [data-testid="stTabsContent"] {
  background: transparent !important;
  border: none !important;
  box-shadow: none !important;
  padding: .75rem 0 0 !important;
}

[data-testid="stMetricLabel"] p {
  font-size: .75rem !important;
  font-weight: 600 !important;
  letter-spacing: .1em !important;
  text-transform: uppercase !important;
  color: var(--t3) !important;
}
[data-testid="stMetricValue"] {
  font-size: 2rem !important;
  font-weight: 700 !important;
  background: linear-gradient(135deg,#fff 0%,#94a3b8 100%) !important;
  -webkit-background-clip: text !important;
  -webkit-text-fill-color: transparent !important;
  background-clip: text !important;
}

/* =====
  BUTTONS
  ===== */
.stButton > button {
  background: linear-gradient(135deg, rgba(59,130,246,.8), rgba(139,92,246,.8)) !important;
  color: #fff !important;
  border: 1px solid rgba(255,255,255,.2) !important;
  border-radius: var(--rs) !important;
  padding: .65rem 1.4rem !important;
  font-family: 'Sora', sans-serif !important;
  font-weight: 600 !important;
  font-size: .875rem !important;
  letter-spacing: .02em !important;
  backdrop-filter: blur(12px) !important;

```

```

    box-shadow: 0 4px 24px rgba(59,130,246,.3), inset 0 1px 0 rgba(255,255,255,.2) !important;
    transition: all .25s cubic-bezier(.16,1,.3,1) !important;
    position: relative !important;
    overflow: hidden !important;
}
.stButton > button:hover {
    transform: translateY(-2px) scale(1.02) !important;
    box-shadow: 0 8px 32px rgba(59,130,246,.5), inset 0 1px 0 rgba(255,255,255,.3) !important;
    border-color: rgba(255,255,255,.35) !important;
}
.stButton > button:active {
    transform: translateY(0) scale(.98) !important;
}

/* =====
   INPUTS
   ===== */

.stTextInput > div > div > input,
.stNumberInput > div > div > input,
.stTextArea > div > div > textarea {
    background: rgba(255,255,255,.05) !important;
    border: 1px solid rgba(255,255,255,.12) !important;
    border-radius: var(--rs) !important;
    color: var(--t1) !important;
    font-family: 'Sora', sans-serif !important;
    font-size: .9rem !important;
    padding: .65rem 1rem !important;
    transition: all .25s ease !important;
    box-shadow: inset 0 1px 4px rgba(0,0,0,.2) !important;
}
.stTextInput > div > div > input:focus,
.stNumberInput > div > div > input:focus,
.stTextArea > div > div > textarea:focus {
    border-color: rgba(59,130,246,.7) !important;
    box-shadow: 0 0 3px rgba(59,130,246,.15), inset 0 1px 4px rgba(0,0,0,.2) !important;
    outline: none !important;
}

/* =====
   SELECTBOX
   ===== */

.stSelectbox > div > div {
    background: rgba(255,255,255,.05) !important;
    border: 1px solid rgba(255,255,255,.12) !important;
    border-radius: var(--rs) !important;
    color: var(--t1) !important;
}
.stSelectbox > div > div > div { color: var(--t1) !important; }

[data-baseweb="popover"] {
    background: rgba(15,15,30,.97) !important;
    backdrop-filter: blur(24px) !important;
    border: 1px solid rgba(255,255,255,.12) !important;
    border-radius: var(--rs) !important;

```

```

    box-shadow: 0 24px 48px rgba(0,0,0,.6) !important;
}
[data-baseweb="menu"] li {
    background: transparent !important;
    color: var(--t2) !important;
    transition: background .15s ease !important;
}
[data-baseweb="menu"] li:hover {
    background: rgba(59,130,246,.15) !important;
    color: #fff !important;
}

/* =====
TABS
===== */

.stTabs [data-baseweb="tab-list"] {
    background: rgba(255,255,255,.05) !important;
    border-radius: 14px !important;
    padding: 4px !important;
    gap: 4px !important;
    border: 1px solid rgba(255,255,255,.08) !important;
}
.stTabs [data-baseweb="tab"] {
    background: transparent !important;
    border-radius: 10px !important;
    color: var(--t3) !important;
    font-weight: 500 !important;
    font-size: .875rem !important;
    transition: all .2s ease !important;
    padding: .5rem 1.2rem !important;
}
.stTabs [aria-selected="true"] {
    background: rgba(255,255,255,.12) !important;
    color: #fff !important;
    box-shadow: 0 2px 12px rgba(0,0,0,.3), inset 0 1px 0 rgba(255,255,255,.2) !important;
}

/* =====
BUG FIX 4b and 4c
DATAFRAME — blank rows + jumbled menu text

b) No overflow:hidden anywhere near a dataframe.
    The GlideDataEditor canvas paints table rows
    at a position that can exceed the container
    bounds. overflow:hidden clips those rows,
    leaving a blank white area.

c) Jumbled column-menu text: Streamlit's column
    header context menu uses Material Icons.
    Our font override must NOT reach those elements.
    We only style the outer frame here; the inner
    canvas and popover menus are left untouched.
===== */
.stDataFrame,

```

```

[data-testid="stDataFrame"],
[data-testid="stDataFrameResizable"] {
  border-radius: var(--r) !important;
  /* NO overflow:hidden — it blanks out the rows */
  border: 1px solid rgba(255,255,255,.08) !important;
  background: rgba(255,255,255,.03) !important;
}

/* =====
SLIDER
===== */
.stSlider [data-baseweb="slider"] { padding: .5rem 0 !important; }
.stSlider [data-baseweb="slider"] [role="slider"] {
  background: linear-gradient(135deg, var(--blue), var(--purple)) !important;
  border: 2px solid rgba(255,255,255,.3) !important;
  box-shadow: 0 0 12px rgba(59,130,246,.5) !important;
  width: 20px !important; height: 20px !important;
}

/* =====
RADIO (sidebar nav)
===== */
.stRadio > div { display:flex !important; flex-direction:column !important; gap:4px !important; }
.stRadio label {
  background: transparent !important;
  border: 1px solid transparent !important;
  border-radius: var(--rs) !important;
  padding: .6rem 1rem !important;
  cursor: pointer !important;
  transition: all .2s cubic-bezier(.16,1,.3,1) !important;
  display: flex !important;
  align-items: center !important;
  gap: .5rem !important;
  color: var(--t2) !important;
  font-weight: 500 !important;
  font-size: .875rem !important;
}
.stRadio label:hover {
  background: rgba(255,255,255,.07) !important;
  border-color: rgba(255,255,255,.1) !important;
  color: #fff !important;
}
.stRadio label:has(input:checked) {
  background: linear-gradient(135deg, rgba(59,130,246,.2), rgba(139,92,246,.2)) !important;
  border-color: rgba(59,130,246,.4) !important;
  color: #fff !important;
  box-shadow: 0 0 20px rgba(59,130,246,.1) !important;
}
.stRadio [data-baseweb="radio"] > div:first-child { display:none !important; }

/* =====
ALERTS
===== */
[data-testid="stAlert"] {

```

```
border-radius: var(--rs) !important;
backdrop-filter: blur(12px) !important;
border: 1px solid !important;
}
```

```
/* =====
DOWNLOAD BUTTON
===== */
```

```
.stDownloadButton > button {
  background: rgba(16,185,129,.15) !important;
  border: 1px solid rgba(16,185,129,.4) !important;
  color: #10b981 !important;
  border-radius: var(--rs) !important;
  font-weight: 600 !important;
  font-size: .875rem !important;
  transition: all .25s ease !important;
}
.stDownloadButton > button:hover {
  background: rgba(16,185,129,.25) !important;
  transform: translateY(-2px) !important;
  box-shadow: 0 8px 24px rgba(16,185,129,.25) !important;
}
```

```
/* =====
DIVIDER / SCROLLBAR / PLOTLY
===== */
```

```
hr {
  border: none !important;
  border-top: 1px solid rgba(255,255,255,.08) !important;
  margin: 1.5rem 0 !important;
}
::-webkit-scrollbar { width:6px; height:6px; }
::-webkit-scrollbar-track { background: rgba(255,255,255,.02); }
::-webkit-scrollbar-thumb { background: rgba(255,255,255,.12); border-radius:99px; }
::-webkit-scrollbar-thumb:hover { background: rgba(255,255,255,.22); }
.js-plotly-plot { border-radius: var(--r) !important; overflow:hidden !important; }
```

```
/* =====
CUSTOM HTML COMPONENTS
===== */
```

```
/* Sidebar brand */
.sidebar-brand {
  display: flex; align-items: center; gap: 10px;
  padding: .5rem 0 1.5rem;
}
.sidebar-brand-icon {
  width:38px; height:38px;
  background: linear-gradient(135deg,#3b82f6,#8b5cf6);
  border-radius:10px;
  display:flex; align-items:center; justify-content:center;
  font-size:1.2rem;
  box-shadow: 0 4px 16px rgba(59,130,246,.4);
}
```



```

.sidebar-nav-label {
  font-size:.65rem; letter-spacing:.12em;
  text-transform:uppercase;
  color: rgba(255,255,255,.25) !important;
  padding:1rem .25rem .4rem; font-weight:600;
}

/* Page chip */
.page-chip {
  display:inline-block;
  padding:.25rem .75rem;
  background: linear-gradient(135deg,rgba(59,130,246,.2),rgba(139,92,246,.2));
  border: 1px solid rgba(59,130,246,.35);
  border-radius:99px;
  font-size:.7rem; font-weight:700; letter-spacing:.1em;
  text-transform:uppercase; color:#93c5fd !important;
}

/* Stat card */
.stat-card {
  background: var(--glass-bg);
  backdrop-filter: var(--blur);
  -webkit-backdrop-filter: var(--blur);
  border: 1px solid var(--glass-bdr);
  border-radius: var(--r);
  box-shadow: var(--glass-shd), inset 0 1px 0 rgba(255,255,255,.1);
  padding: 1.5rem;
  position: relative;
  overflow: hidden; /* stat-card CAN have overflow:hidden — no dataframe inside */
  transition: all .35s cubic-bezier(.16,1,.3,1);
}
.stat-card::before {
  content:"";
  position:absolute; top:0; left:0; right:0; height:1px;
  background: linear-gradient(90deg,transparent,rgba(255,255,255,.25),transparent);
}
.stat-card:hover {
  transform: translateY(-4px);
  box-shadow: var(--glass-shd), 0 24px 64px rgba(59,130,246,.18), inset 0 1px 0 rgba(255,255,255,.18);
}
.stat-card .glow {
  position:absolute; bottom:-20px; right:-20px;
  width:80px; height:80px; border-radius:50%;
  opacity:.3; filter:blur(20px);
}

/* Glass section wrapper
IMPORTANT: deliberately NO overflow:hidden here.
overflow:hidden clips the GlideDataEditor canvas
that renders st.dataframe row content. */
.glass-section {
  background: var(--glass-bg);
  backdrop-filter: var(--blur);
  -webkit-backdrop-filter: var(--blur);

```

```

border: 1px solid var(--glass-bdr);
border-radius: var(--r);
box-shadow: var(--glass-shd), inset 0 1px 0 rgba(255,255,255,.08);
padding: 1.75rem;
margin-bottom: 1rem;
position: relative;
}
.glass-section::before {
  content:"";
  position:absolute; top:0; left:0; right:0; height:1px;
  background: linear-gradient(90deg,transparent,rgba(255,255,255,.2),transparent);
  pointer-events:none;
}

/* Pulse animation */
@keyframes pulse {
  0%,100% { opacity:1; transform:scale(1); }
  50%    { opacity:.6; transform:scale(.85); }
}

/* Badges */
.badge-blue { background:rgba(59,130,246,.2); border:1px solid rgba(59,130,246,.4); color:#93c5fd
!important; display:inline-block; padding:.2rem .6rem; border-radius:99px; font-size:.7rem; font-
weight:700; }
.badge-green { background:rgba(16,185,129,.2); border:1px solid rgba(16,185,129,.4); color:#6ee7b7
!important; display:inline-block; padding:.2rem .6rem; border-radius:99px; font-size:.7rem; font-
weight:700; }
.badge-pink { background:rgba(236,72,153,.2); border:1px solid rgba(236,72,153,.4); color:#f9a8d4
!important; display:inline-block; padding:.2rem .6rem; border-radius:99px; font-size:.7rem; font-
weight:700; }
</style>

<!-- Animated background orbs -->
<div style="position:fixed;top:0;left:0;width:100vw;height:100vh;pointer-events:none;z-
index:0;overflow:hidden;">
  <div style="position:absolute;top:5%;left:10%;width:500px;height:500px;
    background:radial-gradient(circle,rgba(59,130,246,.18) 0%,transparent 70%);
    border-radius:50%;animation:orb1 18s ease-in-out infinite;"></div>
  <div style="position:absolute;top:50%;right:8%;width:420px;height:420px;
    background:radial-gradient(circle,rgba(139,92,246,.16) 0%,transparent 70%);
    border-radius:50%;animation:orb2 24s ease-in-out infinite;"></div>
  <div style="position:absolute;bottom:10%;left:30%;width:380px;height:380px;
    background:radial-gradient(circle,rgba(6,182,212,.14) 0%,transparent 70%);
    border-radius:50%;animation:orb3 20s ease-in-out infinite;"></div>
  <div style="position:absolute;top:30%;left:55%;width:300px;height:300px;
    background:radial-gradient(circle,rgba(236,72,153,.10) 0%,transparent 70%);
    border-radius:50%;animation:orb1 28s ease-in-out infinite reverse;"></div>
</div>
""",
  unsafe_allow_html=True,
)

# _____
# Session state

```

```

# -----
if "logged_in" not in st.session_state:
    st.session_state.logged_in = False
if "username" not in st.session_state:
    st.session_state.username = ""

# -----
# ML model loader
# -----
@st.cache_resource
def load_models():
    try:
        dm = joblib.load("demand_model.joblib")
        pm = joblib.load("price_model.joblib")
        return dm, pm
    except Exception:
        return None, None

demand_model_obj, price_model_obj = load_models()

# -----
# Plotly theme
# -----
PLOTLY_THEME = dict(
    template="plotly_dark",
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",
    font=dict(family="Sora, sans-serif", color="rgba(255,255,255,.7)", size=12),
    margin=dict(l=20, r=20, t=40, b=20),
)
COLOR_SEQ = ["#3b82f6", "#8b5cf6", "#06b6d4", "#10b981", "#f59e0b", "#ec4899", "#f97316"]

# =====
# LOGIN PAGE
# =====
def login_page():
    st.markdown(
        """
        <div style="text-align:center;padding:3rem 0 2rem;">
        <div style="display:inline-flex;align-items:center;justify-content:center;
            width:64px;height:64px;
            background:linear-gradient(135deg,#3b82f6,#8b5cf6);
            border-radius:18px;margin-bottom:1.25rem;
            box-shadow:0 12px 40px rgba(59,130,246,.45);">
            <span style="font-size:2rem;color:#fff;">◆</span>
        </div>
        <h1 style="font-size:2.6rem!important;font-weight:800!important;
            background:linear-gradient(135deg,#fff 0%,#94a3b8 100%);
            -webkit-background-clip:text;-webkit-text-fill-color:transparent;
            background-clip:text;margin:0;letter-spacing:-.04em;">
            FMCG Nexus
        </h1>
        <p style="color:rgba(255,255,255,.4)!important;font-size:.95rem;
            margin-top:.4rem;font-weight:300;">

```

Intelligent Analytics & Commerce Platform

</p>

</div>

""",

unsafe_allow_html=True,

)

_, col_m, _ = st.columns([1, 1.6, 1])

with col_m:

st.markdown('<div class="glass-section">', unsafe_allow_html=True)

t1, t2 = st.tabs(["● Sign In", "© Create Account"])

with t1:

st.markdown("
", unsafe_allow_html=True)

with st.form("login_form", border=False):

uname = st.text_input("Username", placeholder="Enter your username", key="li_u")

pwd = st.text_input("Password", type="password", placeholder="●●●●●●●●", key="li_p")

st.markdown("
", unsafe_allow_html=True)

submitted = st.form_submit_button("Sign In →", use_container_width=True)

if submitted:

user = db.authenticate_user(uname, pwd)

if user:

st.session_state.logged_in = True

st.session_state.username = uname

st.rerun()

else:

st.error("Invalid credentials. Please try again.")

with t2:

st.markdown("
", unsafe_allow_html=True)

with st.form("register_form", border=False):

rn = st.text_input("Full Name", placeholder="Jane Doe", key="rg_n")

rph = st.text_input("Phone Number", placeholder="+91 98765 43210", key="rg_ph")

rem = st.text_input("Email Address", placeholder="jane@company.com", key="rg_e")

ru = st.text_input("Username", placeholder="Choose a username", key="rg_u")

rp = st.text_input("Password", type="password",
placeholder="Min. 8 characters", key="rg_p")

st.markdown("
", unsafe_allow_html=True)

submitted2 = st.form_submit_button("Create Account →", use_container_width=True)

if submitted2:

if not all([rn, rph, rem, ru, rp]):

st.warning("Please complete all fields to continue.")

else:

ok, msg = db.register_user(rn, rph, rem, ru, rp)

if ok:

st.success(msg + " You can now sign in.")

else:

st.error(msg)

st.markdown("</div>", unsafe_allow_html=True)

def logout():

st.session_state.logged_in = False

st.session_state.username = ""

```
st.rerun()
```

```
# =====
# SIDEBAR
# =====
def render_sidebar():
    with st.sidebar:
        initial = st.session_state.username[0].upper() if st.session_state.username else "?"
        st.markdown(
            f"""
            <div class="sidebar-brand">
                <div class="sidebar-brand-icon">
                    <span style="color:#fff;font-size:1.2rem;">◆</span>
                </div>
                <div>
                    <div style="font-weight:700;font-size:1rem;color:#fff;letter-spacing:-.02em;">
                        FMCG Nexus
                    </div>
                    <div style="font-size:.7rem;color:rgba(255,255,255,.35);font-weight:400;">
                        Analytics Platform
                    </div>
                </div>
            </div>

            <div style="background:rgba(255,255,255,.05);border:1px solid rgba(255,255,255,.08);
                border-radius:12px;padding:.65rem 1rem;margin-bottom:1.25rem;
                display:flex;align-items:center;gap:10px;">
                <div style="width:32px;height:32px;
                    background:linear-gradient(135deg,#3b82f6,#8b5cf6);
                    border-radius:8px;display:flex;align-items:center;
                    justify-content:center;font-size:.9rem;font-weight:700;
                    color:#fff;flex-shrink:0;">
                    {initial}
                </div>
                <div>
                    <div style="font-size:.8rem;font-weight:600;color:rgba(255,255,255,.9);">
                        {st.session_state.username}
                    </div>
                    <div style="font-size:.65rem;color:rgba(255,255,255,.35);">
                        Active session
                    </div>
                </div>
            </div>

            <div class="sidebar-nav-label">Navigation</div>
            """,
            unsafe_allow_html=True,
        )

    nav = {
        "Dashboard": "◆ Dashboard",
        "Product Management": "▣ Products",
        "Sales": "⊕ Sales",
        "Predictions": "⦿ AI Predictions",
```

```

    "Analytics Filter": ":: Analytics",
}
selection = st.radio(
    "Navigate",
    options=list(nav.keys()),
    format_func=lambda x: nav[x],
    label_visibility="collapsed",
)

st.markdown("<br>", unsafe_allow_html=True)
st.markdown(
    """
    <div style="font-size:.65rem;letter-spacing:.1em;text-transform:uppercase;
        color:rgba(255,255,255,.2);padding:0 .25rem .4rem;font-weight:600;">
        System
    </div>
    """,
    unsafe_allow_html=True,
)
if st.button("⊞ Sign Out", use_container_width=True):
    logout()

st.markdown(
    """
    <div style="margin-top:2rem;padding:.75rem;
        background:rgba(255,255,255,.03);
        border:1px solid rgba(255,255,255,.06);
        border-radius:12px;text-align:center;">
        <div style="font-size:.65rem;color:rgba(255,255,255,.2);font-weight:500;">
            FMCG Nexus · v2.0
        </div>
    </div>
    """,
    unsafe_allow_html=True,
)

```

```

return selection

```

```

# PAGE HEADER
#
def page_header(icon, label, chip=""):
    chip_html = f'<span class="page-chip">{chip}</span>' if chip else ""
    st.markdown(
        f"""
        <div style="margin-bottom:1.75rem;">
            {chip_html}
            <h1 style="font-size:2.2rem!important;font-weight:800!important;
                background:linear-gradient(135deg,#fff 0%,#94a3b8 100%);
                -webkit-background-clip:text;-webkit-text-fill-color:transparent;
                background-clip:text;margin:.2rem 0 .3rem;letter-spacing:-.04em;">
                {icon}&nbsp; {label}
            </h1>
        </div>
        """
    )

```

```

        """
        unsafe_allow_html=True,
    )

# =====
# DASHBOARD
# =====
def display_dashboard():
    page_header("📊", "Dashboard", chip="Overview")

    products = db.get_all_products()
    sales = db.get_all_sales()
    total_prods = len(products)
    total_sales = len(sales)
    total_rev = sum(s["total_price"] for s in sales)
    avg_order = total_rev / total_sales if total_sales else 0

    c1, c2, c3, c4 = st.columns(4)
    cards = [
        (c1, "Total Products", str(total_prods), "#3b82f6", "📦"),
        (c2, "Total Transactions", str(total_sales), "#8b5cf6", "📈"),
        (c3, "Total Revenue", f"₹{total_rev:,.0f}", "#06b6d4", "💰"),
        (c4, "Avg Order Value", f"₹{avg_order:,.2f}", "#10b981", "🟢"),
    ]
    for col, lbl, val, color, ico in cards:
        with col:
            st.markdown(
                f"""
                <div class="stat-card">
                <div class="glow" style="background:{color};"></div>
                <div style="display:flex;justify-content:space-between;align-items:flex-start;">
                <div>
                <div style="font-size:.7rem;letter-spacing:.1em;text-transform:uppercase;
                color:rgba(255,255,255,.35);font-weight:600;margin-bottom:.6rem;">
                    {lbl}
                </div>
                <div style="font-size:1.85rem;font-weight:800;
                background:linear-gradient(135deg,#fff 0%,#94a3b8 100%);
                -webkit-background-clip:text;-webkit-text-fill-color:transparent;
                background-clip:text;line-height:1;">
                    {val}
                </div>
                </div>
                <div style="width:40px;height:40px;background:rgba(255,255,255,.07);
                border:1px solid rgba(255,255,255,.12);border-radius:10px;
                display:flex;align-items:center;justify-content:center;
                font-size:1.1rem;color:{color};">
                    {ico}
                </div>
                </div>
                </div>
                """
            )
    """
    unsafe_allow_html=True,
    )

```

```

st.markdown("<br>", unsafe_allow_html=True)

if total_sales > 0:
    df_s = pd.DataFrame(sales)
    df_s["date"] = pd.to_datetime(df_s["date"]).dt.date
    rev_trend = df_s.groupby("date")["total_price"].sum().reset_index()

    ca, cb = st.columns([2, 1])
    with ca:
        st.markdown('<div class="glass-section">', unsafe_allow_html=True)
        st.markdown("#### Revenue Timeline")
        fig = go.Figure()
        fig.add_trace(go.Scatter(
            x=rev_trend["date"], y=rev_trend["total_price"],
            mode="lines", line=dict(color="#3b82f6", width=2.5, shape="spline"),
            fill="tozero", fillcolor="rgba(59,130,246,.08)",
        ))
        fig.update_layout(
            **PLOTLY_THEME, height=280, showlegend=False,
            xaxis=dict(showgrid=False, zeroline=False, color="rgba(255,255,255,.3)"),
            yaxis=dict(showgrid=True, gridcolor="rgba(255,255,255,.05)",
                zeroline=False, color="rgba(255,255,255,.3)"),
        )
        st.plotly_chart(fig, use_container_width=True)
        st.markdown("</div>", unsafe_allow_html=True)

    with cb:
        st.markdown('<div class="glass-section">', unsafe_allow_html=True)
        st.markdown("#### Sales by Location")
        if "location" in df_s.columns:
            loc = df_s.groupby("location")["total_price"].sum().reset_index()
            fig2 = px.pie(loc, values="total_price", names="location",
                color_discrete_sequence=COLOR_SEQ, hole=0.55)
            fig2.update_traces(textfont_size=11, marker_line_width=0)
            fig2.update_layout(**PLOTLY_THEME, height=280, showlegend=True,
                legend=dict(orientation="h", y=-0.15, font_size=10))
            st.plotly_chart(fig2, use_container_width=True)
        else:
            st.info("Location data not available.")
            st.markdown("</div>", unsafe_allow_html=True)
    else:
        st.markdown(
            """
            <div class="glass-section" style="text-align:center;padding:3rem;">
            <div style="font-size:3rem;margin-bottom:1rem;opacity:.4;">⊕</div>
            <div style="color:rgba(255,255,255,.4);font-size:.9rem;">
            No sales data yet. Start recording transactions to see live analytics.
            </div>
            </div>
            """,
            unsafe_allow_html=True,
        )

```

```

#
# PRODUCT MANAGEMENT
#
# BUG FIX 4 applied here:
# Old code wrapped metrics + divider + dataframe in ONE
# glass-section div. That created:
# (a) the unwanted ghost glass box above the metrics
# (b) overflow:hidden (if present) blanking the dataframe
#
# Fix: metrics are rendered directly (they already have
# glass styling via [data-testid="stMetric"] CSS).
# Dataframe lives in its own separate glass-section that
# has NO overflow:hidden.
#
def display_products():
    page_header("🏠", "Products", chip="Inventory")

    tab1, tab2 = st.tabs(["🏠 All Products", "+ Add Product"])

    with tab1:
        products = db.get_all_products()
        if products:
            df = pd.DataFrame(products)
            total_val = (df["price"] * df["stock"]).sum()
            low_stock = int((df["stock"] < 20).sum())

            # Metrics — NO wrapper div; CSS handles glass styling per card
            mc1, mc2, mc3 = st.columns(3)
            mc1.metric("SKUs in Inventory", len(df))
            mc2.metric("Inventory Value", f"₹{total_val:,.0f}")
            mc3.metric("Low-Stock Alerts", low_stock)

            st.markdown("<br>", unsafe_allow_html=True)

            # Dataframe in its own glass section (separate from metrics)
            # glass-section has NO overflow:hidden → rows render correctly
            st.markdown('<div class="glass-section">', unsafe_allow_html=True)
            st.dataframe(df, use_container_width=True, height=380)
            st.markdown('</div>', unsafe_allow_html=True)
        else:
            st.markdown(
                """
                <div class="glass-section" style="text-align:center;padding:3rem;">
                <div style="font-size:2.5rem;opacity:.3;margin-bottom:1rem;">🏠</div>
                <div style="color:rgba(255,255,255,.35);font-size:.9rem;">
                Your inventory is empty. Add products to get started.
                </div>
                </div>
                """,
                unsafe_allow_html=True,
            )

    with tab2:
        st.markdown('<div class="glass-section">', unsafe_allow_html=True)

```

```

st.markdown("#### New Product Details")
pc1, pc2 = st.columns(2)
with pc1:
    new_name = st.text_input("Product Name", placeholder="e.g. Amul Butter 500g")
    new_price = st.number_input("Price (₹)", min_value=1.0, value=50.0, step=0.5)
with pc2:
    new_cat = st.selectbox(
        "Category",
        ["Dairy", "Bakery", "Personal Care", "Snacks",
         "Home Care", "Beverages", "Groceries", "Other"],
    )
    new_stock = st.number_input("Initial Stock", min_value=1, value=100, step=10)

st.markdown("<br>", unsafe_allow_html=True)
if st.button("⊕ Add to Inventory", use_container_width=True):
    if not new_name:
        st.warning("Product name is required.")
    else:
        ok, msg = db.add_product(new_name, new_cat, new_price, new_stock)
        st.success(msg) if ok else st.error(msg)
        if ok:
            st.rerun()
st.markdown("</div>", unsafe_allow_html=True)

```

```

# SALES

```

```

def display_sales():
    page_header("⊕", "Sales", chip="Transactions")

    products = db.get_all_products()
    if not products:
        st.markdown(
            """
            <div class="glass-section" style="text-align:center;padding:2.5rem;">
            <p style="color:rgba(255,255,255,.4);">
            Add products to inventory before recording sales.
            </p>
            </div>
            """,
            unsafe_allow_html=True,
        )
    return

```

```

df_prods = pd.DataFrame(products)
col1, col2 = st.columns([1, 1.3], gap="large")

```

```

with col1:
    st.markdown('<div class="glass-section">', unsafe_allow_html=True)
    st.markdown("#### Record New Sale")
    sel = st.selectbox("Select Product", df_prods["name"].tolist())
    row = df_prods[df_prods["name"] == sel].iloc[0]
    max_stock = int(row["stock"])
    unit_price = float(row["price"])

```



```

st.markdown("#### Recent Transactions")
sales = db.get_all_sales()
if sales:
    df_s = pd.DataFrame(sales).sort_values("date", ascending=False).head(12)
    st.dataframe(
        df_s[["product", "quantity", "total_price", "location", "date"]],
        use_container_width=True, height=440,
    )
else:
    st.markdown(
        "<div style='text-align:center;padding:2rem;color:rgba(255,255,255,.3);'"
        "font-size:.9rem;'>No transactions yet.</div>",
        unsafe_allow_html=True,
    )
st.markdown("</div>", unsafe_allow_html=True)

```

```

# ML PREDICTIONS
#

```

```

def display_predictions():
    page_header("⦿", "AI Predictions", chip="Machine Learning")

    if not demand_model_obj or not price_model_obj:
        st.markdown(
            """
            <div class="glass-section" style="text-align:center;padding:3rem;">
            <div style="font-size:2.5rem;margin-bottom:1rem;opacity:.4;">⦿</div>
            <p style="color:rgba(239,68,68,.8);font-weight:600;">ML models not found.</p>
            <p style="color:rgba(255,255,255,.35);font-size:.85rem;">
            Run <code style="background:rgba(255,255,255,.1);padding:2px 8px;
            border-radius:6px;">python generate_model.py</code> to train and save models.
            </p>
            </div>
            """,
            unsafe_allow_html=True,
        )
    return

    try:
        df_csv = pd.read_csv("fmcg_sales_data.csv")
        prod_list = df_csv["Product"].unique().tolist()
        cat_list = df_csv["Category"].unique().tolist()
    except Exception:
        prod_list = ["Milk (1L)", "Dove Soap (100g)"]
        cat_list = ["Dairy", "Personal Care"]

    st.markdown('<div class="glass-section">', unsafe_allow_html=True)
    st.markdown("#### Prediction Engine")
    pred_mode = st.selectbox(
        "Prediction Type",
        ["Demand Forecasting", "Price Prediction",
        "Discount / Offer Suggestion", "Combined Prediction"],
    )
    st.markdown("</div>", unsafe_allow_html=True)

```

```
st.markdown("<br>", unsafe_allow_html=True)
```

```
pc1, pc2 = st.columns([1.1, 1], gap="large")
```

```
with pc1:
```

```
    st.markdown('<div class="glass-section">', unsafe_allow_html=True)
    st.markdown("#### Input Parameters")
    in_prod = st.selectbox("Product", prod_list)
    in_cat = st.selectbox("Category", cat_list)
    in_loc = st.selectbox("Location", ["Urban", "Suburban", "Rural"])
    in_month = st.slider("Month", 1, 12, value=datetime.now().month,
                        format="%d", help="Select month for prediction")
    in_price = in_demand = None
    if pred_mode in ["Demand Forecasting", "Combined Prediction"]:
        in_price = st.number_input("Test Price (₹)", min_value=1.0, value=50.0, step=0.5)
    if pred_mode in ["Price Prediction", "Combined Prediction"]:
        in_demand = st.number_input("Target Demand (units)", min_value=10, value=100, step=5)
    run_btn = st.button("⚡ Run Prediction Engine", use_container_width=True)
    st.markdown("</div>", unsafe_allow_html=True)
```

```
with pc2:
```

```
    st.markdown('<div class="glass-section">', unsafe_allow_html=True)
    st.markdown("#### Results")
```

```
    if run_btn:
```

```
        with st.spinner("Computing predictions..."):
            fd = demand_model_obj["features"]
            fp = price_model_obj["features"]
```

```
            inp = {"Month": in_month, "Price": in_price or 50.0, "Demand": in_demand or 100}
```

```
            for f in fd:
```

```
                if f not in inp:
```

```
                    inp[f] = 0
```

```
            inp[f"Product_{in_prod}"] = 1
```

```
            inp[f"Category_{in_cat}"] = 1
```

```
            inp[f"Location_{in_loc}"] = 1
```

```
            if pred_mode in ["Demand Forecasting", "Combined Prediction"]:
```

```
                pred_d = demand_model_obj["model"].predict([[inp[k] for k in fd]])[0]
```

```
            st.markdown(
```

```
                f"""
```

```
                <div style="background:linear-gradient(135deg,rgba(59,130,246,.15),
                    rgba(139,92,246,.15));border:1px solid rgba(59,130,246,.3);
                    border-radius:14px;padding:1.25rem;margin-bottom:.75rem;
                    text-align:center;">
```

```
                <div style="font-size:.65rem;text-transform:uppercase;letter-spacing:.12em;
                    color:rgba(255,255,255,.35);margin-bottom:4px;">Predicted Demand</div>
```

```
                <div style="font-size:2.4rem;font-weight:800;
                    background:linear-gradient(135deg,#60a5fa,#a78bfa);
                    -webkit-background-clip:text;-webkit-text-fill-color:transparent;">
```

```
                    {int(pred_d)} units
```

```
                </div>
```

```
            </div>
```

```
            """,
```

```
            ,
```



```

    )
    st.markdown("</div>", unsafe_allow_html=True)

# =====
# ANALYTICS
# =====
def display_analytics():
    page_header("::", "Analytics", chip="Reports")

    sales = db.get_all_sales()
    if not sales:
        st.markdown(
            """
            <div class="glass-section" style="text-align:center;padding:3rem;">
            <div style="font-size:2.5rem;opacity:.3;">::</div>
            <p style="color:rgba(255,255,255,.35);margin-top:1rem;font-size:.9rem;">
            No sales data available for analysis.
            </p>
            </div>
            """,
            unsafe_allow_html=True,
        )
    return

df_sales = pd.DataFrame(sales)
tab1, tab2 = st.tabs([":: Visualizations", "↓ Export"])

with tab1:
    ca, cb = st.columns(2, gap="large")

    with ca:
        st.markdown('<div class="glass-section">', unsafe_allow_html=True)
        st.markdown("#### Revenue by Category")
        cat_s = df_sales.groupby("category")["total_price"].sum().reset_index()
        fig1 = px.pie(cat_s, values="total_price", names="category",
                      color_discrete_sequence=COLOR_SEQ, hole=0.55)
        fig1.update_traces(textfont_size=11, marker_line_width=0)
        fig1.update_layout(**PLOTLY_THEME, height=300,
                           legend=dict(orientation="h", y=-0.2, font_size=10))
        st.plotly_chart(fig1, use_container_width=True)
        st.markdown("</div>", unsafe_allow_html=True)

    with cb:
        st.markdown('<div class="glass-section">', unsafe_allow_html=True)
        st.markdown("#### Top Products by Units Sold")
        ps = (df_sales.groupby("product")["quantity"].sum()
              .reset_index().sort_values("quantity", ascending=True).tail(8))
        fig2 = px.bar(ps, x="quantity", y="product", orientation="h",
                      color="quantity",
                      color_continuous_scale=["#1e3a5f", "#3b82f6", "#8b5cf6"],
                      text="quantity")
        fig2.update_traces(textposition="outside", textfont_size=10,
                           marker_line_width=0, opacity=0.9)
        fig2.update_layout(

```

```

        **PLOTLY_THEME, height=300, showlegend=False, coloraxis_showscale=False,
        xaxis=dict(showgrid=False, zeroline=False, color="rgba(255,255,255,.3)",
        yaxis=dict(showgrid=False, zeroline=False, color="rgba(255,255,255,.4)",
            tickfont=dict(size=10)),
    )
    st.plotly_chart(fig2, use_container_width=True)
    st.markdown("</div>", unsafe_allow_html=True)

```

if "date" in df_sales.columns:

```

    st.markdown('<div class="glass-section">', unsafe_allow_html=True)
    st.markdown("#### Revenue Over Time")
    df_sales["date_only"] = pd.to_datetime(df_sales["date"]).dt.date
    trend = df_sales.groupby("date_only")["total_price"].sum().reset_index()
    fig3 = go.Figure()
    fig3.add_trace(go.Scatter(
        x=trend["date_only"], y=trend["total_price"],
        mode="lines+markers",
        line=dict(color="#3b82f6", width=2.5, shape="spline"),
        marker=dict(size=6, color="#8b5cf6", line=dict(width=1.5, color="#fff")),
        fill="tozeroy", fillcolor="rgba(59,130,246,.07)",
    ))
    fig3.update_layout(
        **PLOTLY_THEME, height=260, showlegend=False,
        xaxis=dict(showgrid=False, zeroline=False, color="rgba(255,255,255,.3)",
        yaxis=dict(showgrid=True, gridcolor="rgba(255,255,255,.04)",
            zeroline=False, color="rgba(255,255,255,.3)",
        )
    )
    st.plotly_chart(fig3, use_container_width=True)
    st.markdown("</div>", unsafe_allow_html=True)

```

with tab2:

```

    st.markdown('<div class="glass-section">', unsafe_allow_html=True)
    st.markdown("#### Export Reports")
    st.markdown(
        "<p style='color:rgba(255,255,255,.4);font-size:.88rem;'>"
        "Download data as CSV for use in external BI tools.</p>",
        unsafe_allow_html=True,
    )

```

ec1, ec2 = st.columns(2, gap="large")

with ec1:

```

    st.download_button(
        label="↓ Sales Report (CSV)",
        data=df_sales.to_csv(index=False).encode("utf-8"),
        file_name=f"sales_report_{datetime.now().strftime('%Y%m%d')}.csv",
        mime="text/csv",
        use_container_width=True,
    )

```

with ec2:

```

    prods = db.get_all_products()
    if prods:
        st.download_button(
            label="↓ Inventory Report (CSV)",
            data=pd.DataFrame(prods).to_csv(index=False).encode("utf-8"),
            file_name=f"inventory_report_{datetime.now().strftime('%Y%m%d')}.csv",

```



```
        mime="text/csv",
        use_container_width=True,
    )
    st.markdown("</div>", unsafe_allow_html=True)
```

```
#
```

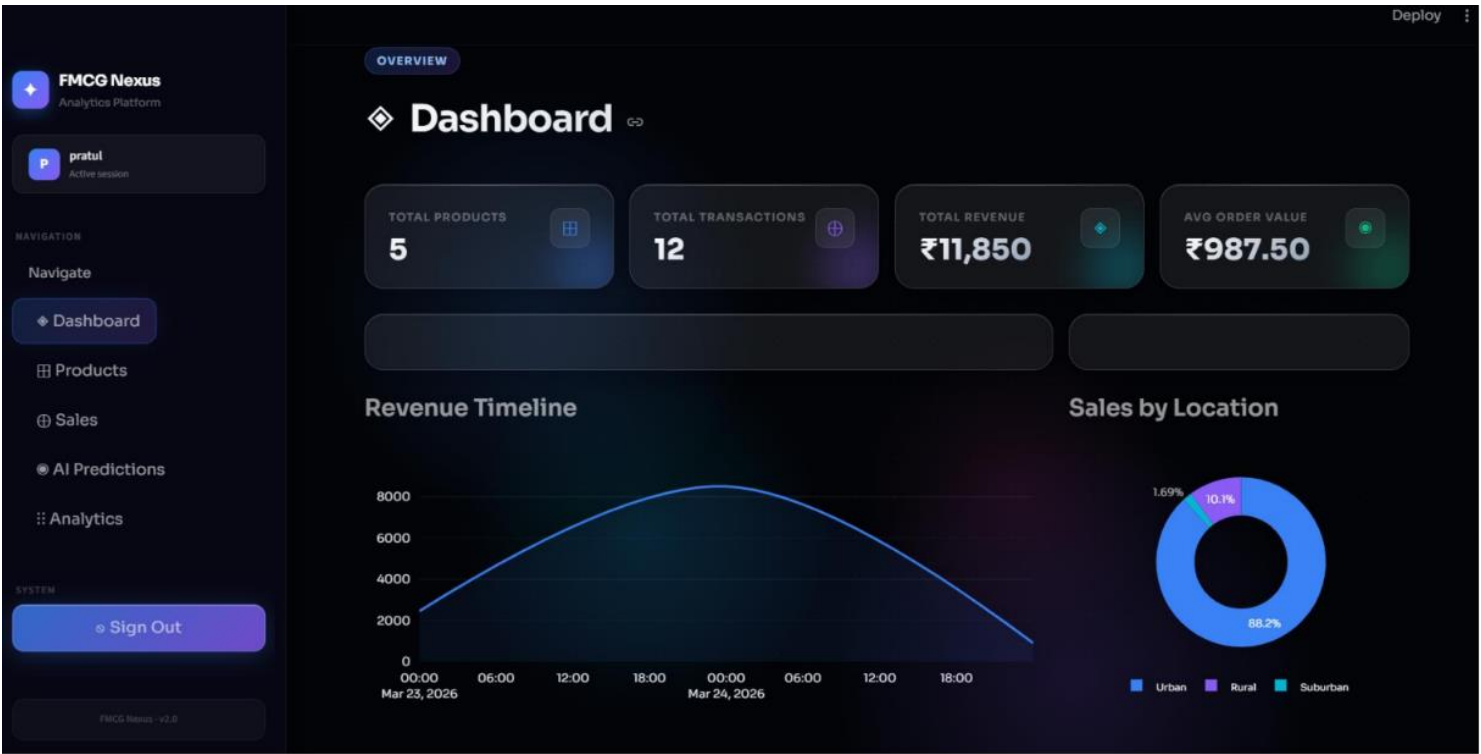
```
# MAIN
```

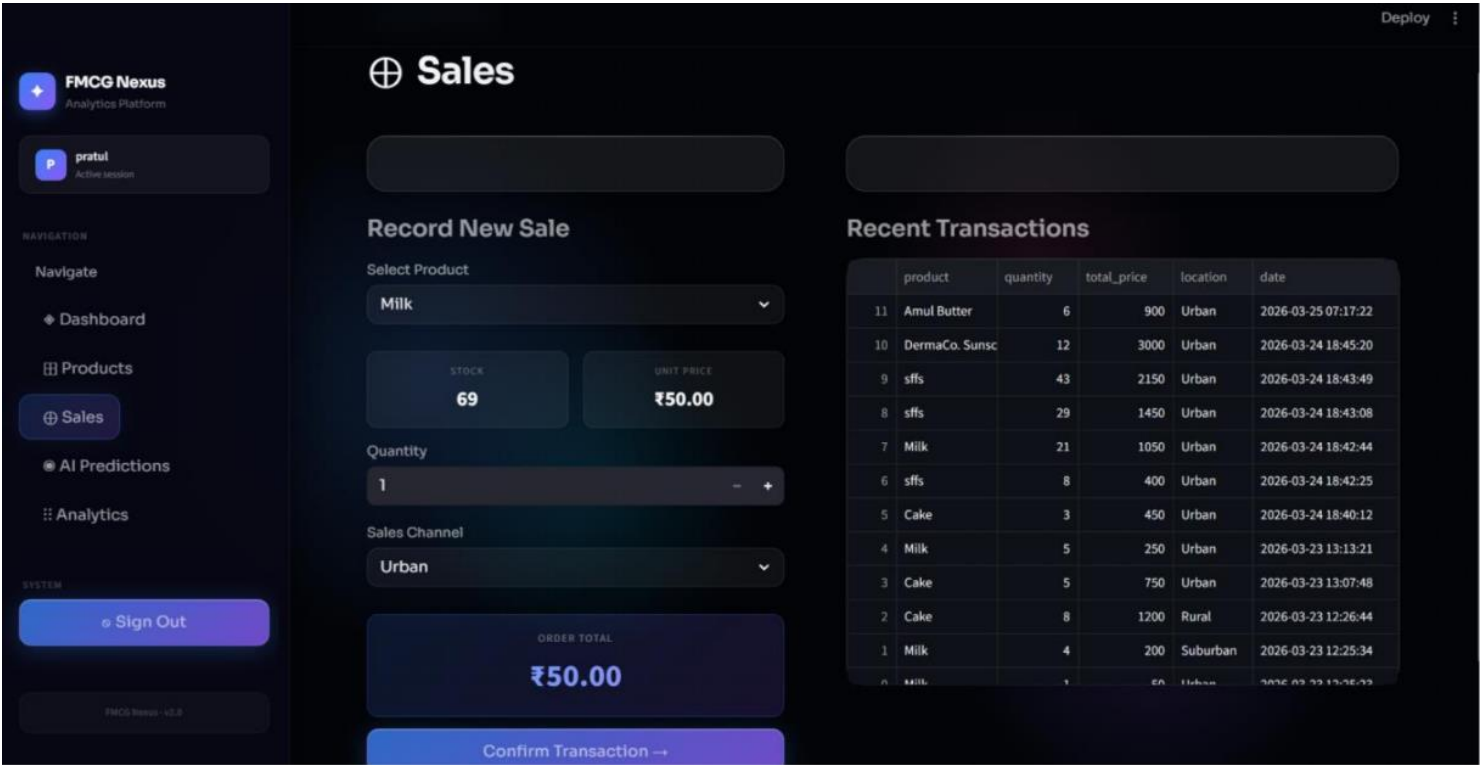
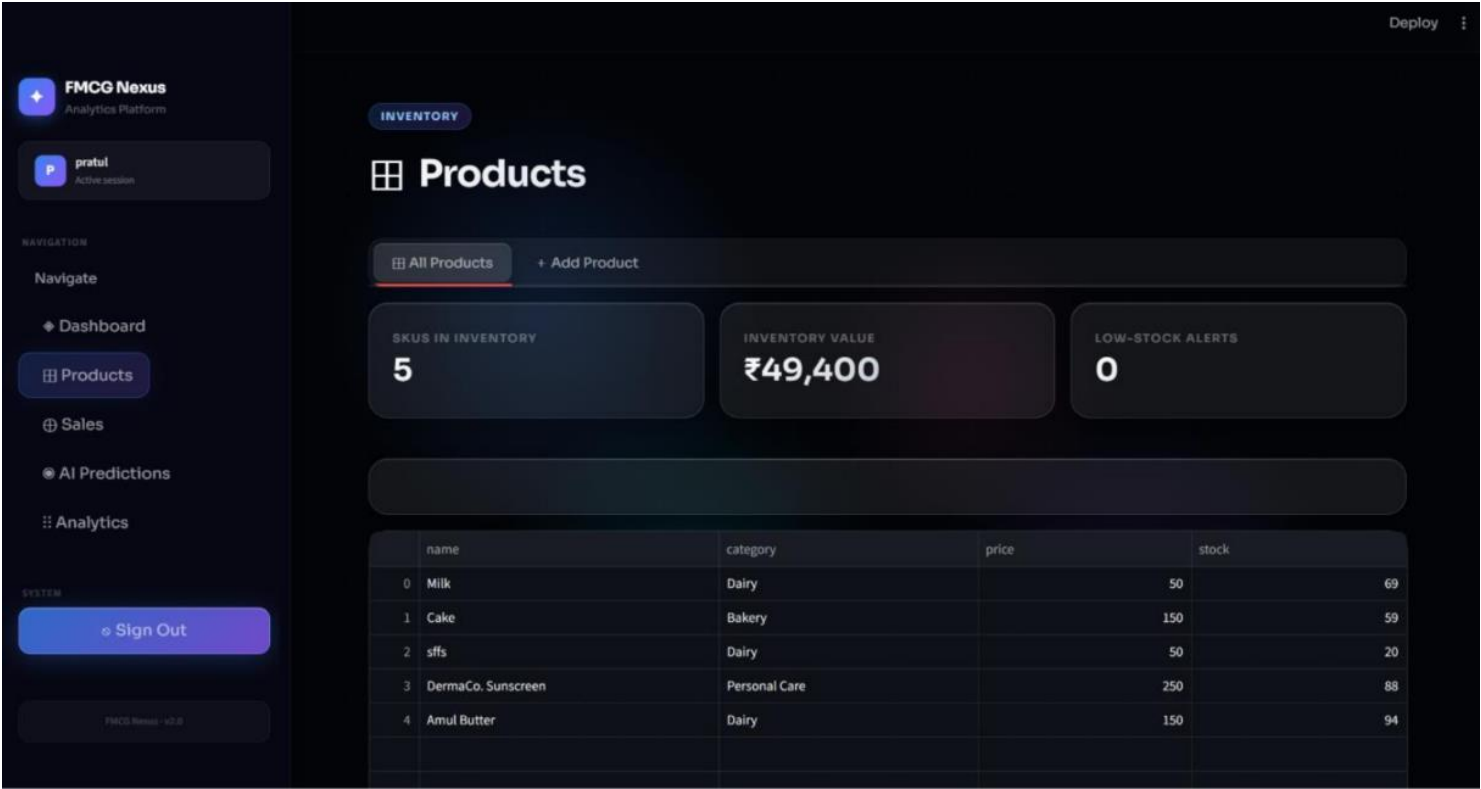
```
#
```

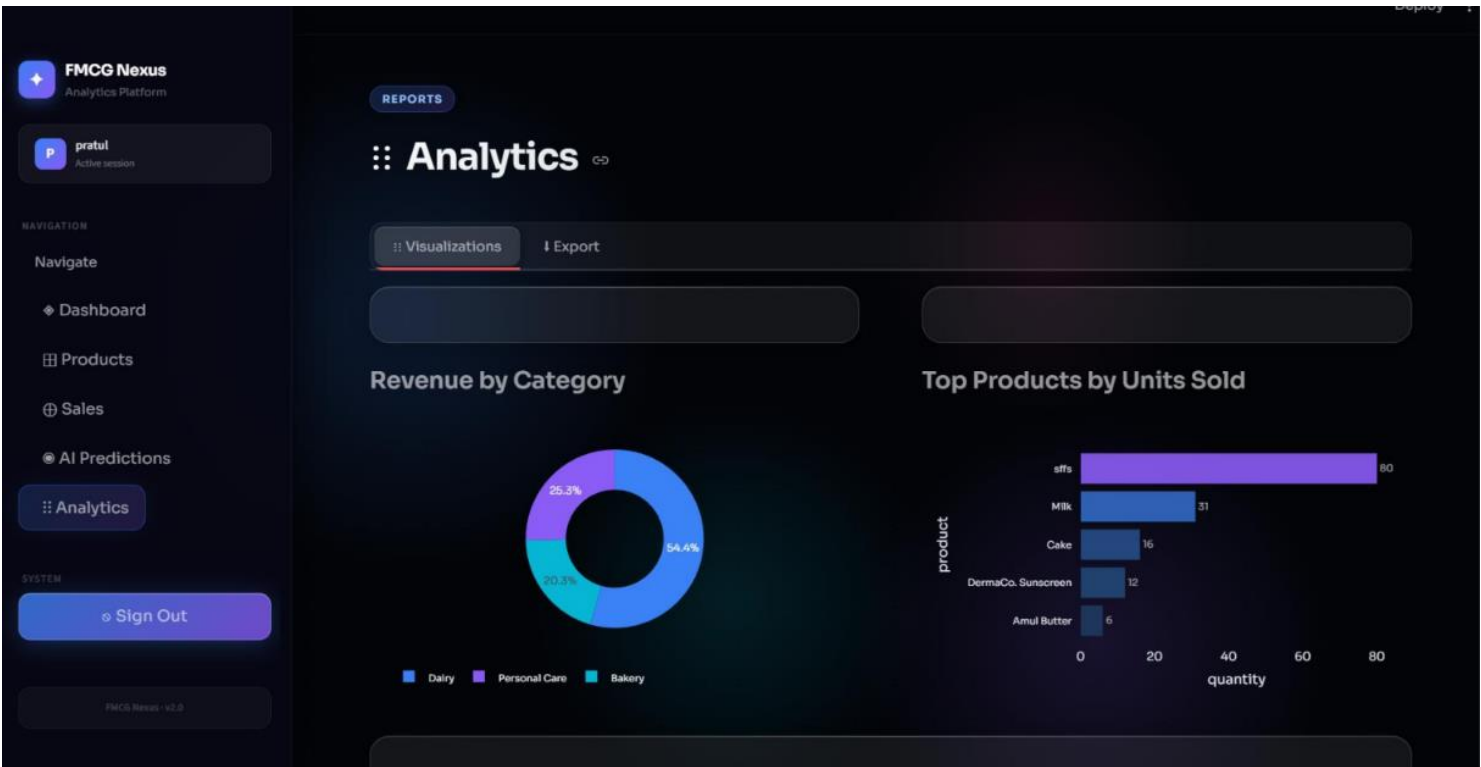
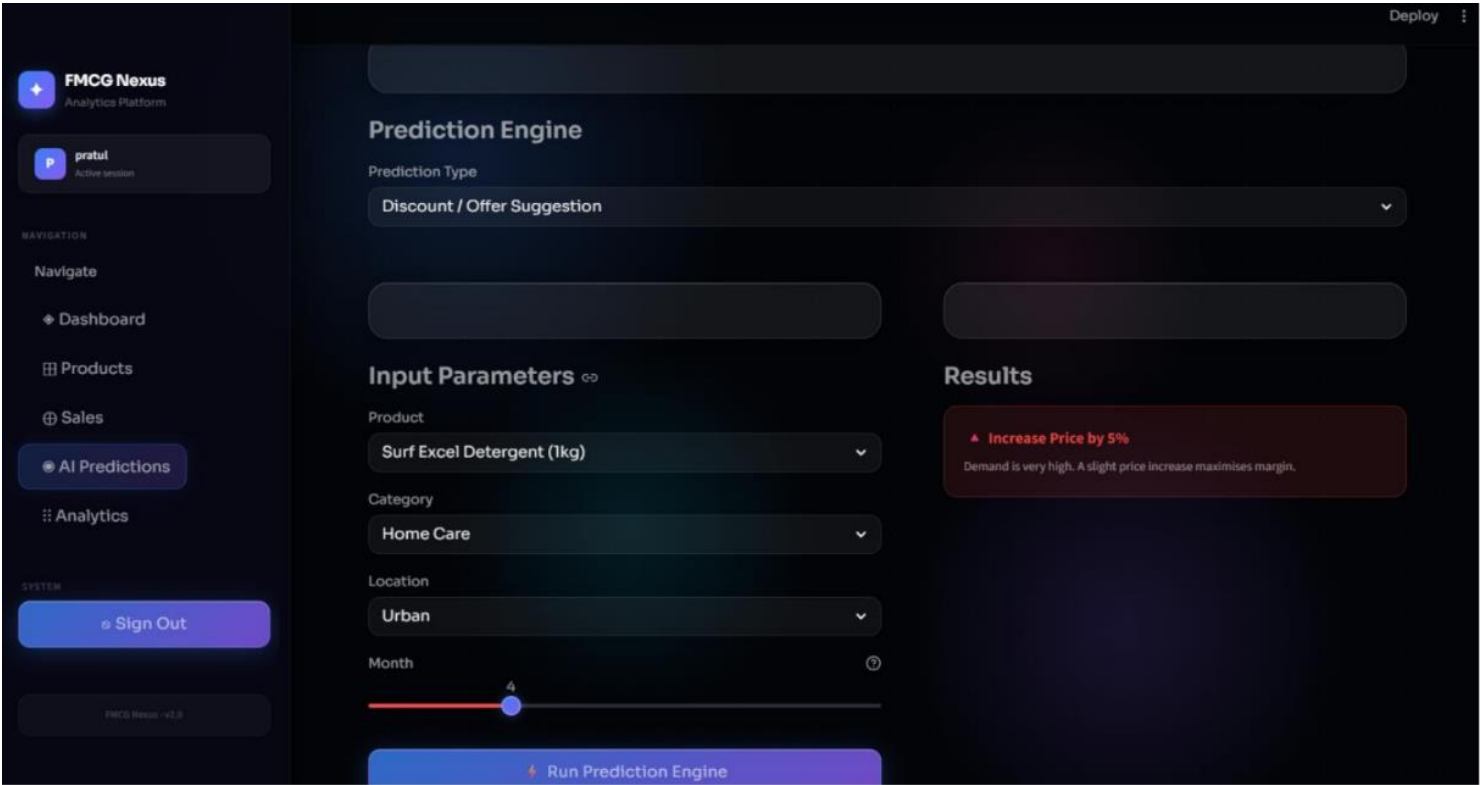
```
def main():
    if not st.session_state.logged_in:
        login_page()
    else:
        page = render_sidebar()
        if page == "Dashboard":    display_dashboard()
        elif page == "Product Management": display_products()
        elif page == "Sales":      display_sales()
        elif page == "Predictions": display_predictions()
        elif page == "Analytics Filter": display_analytics()

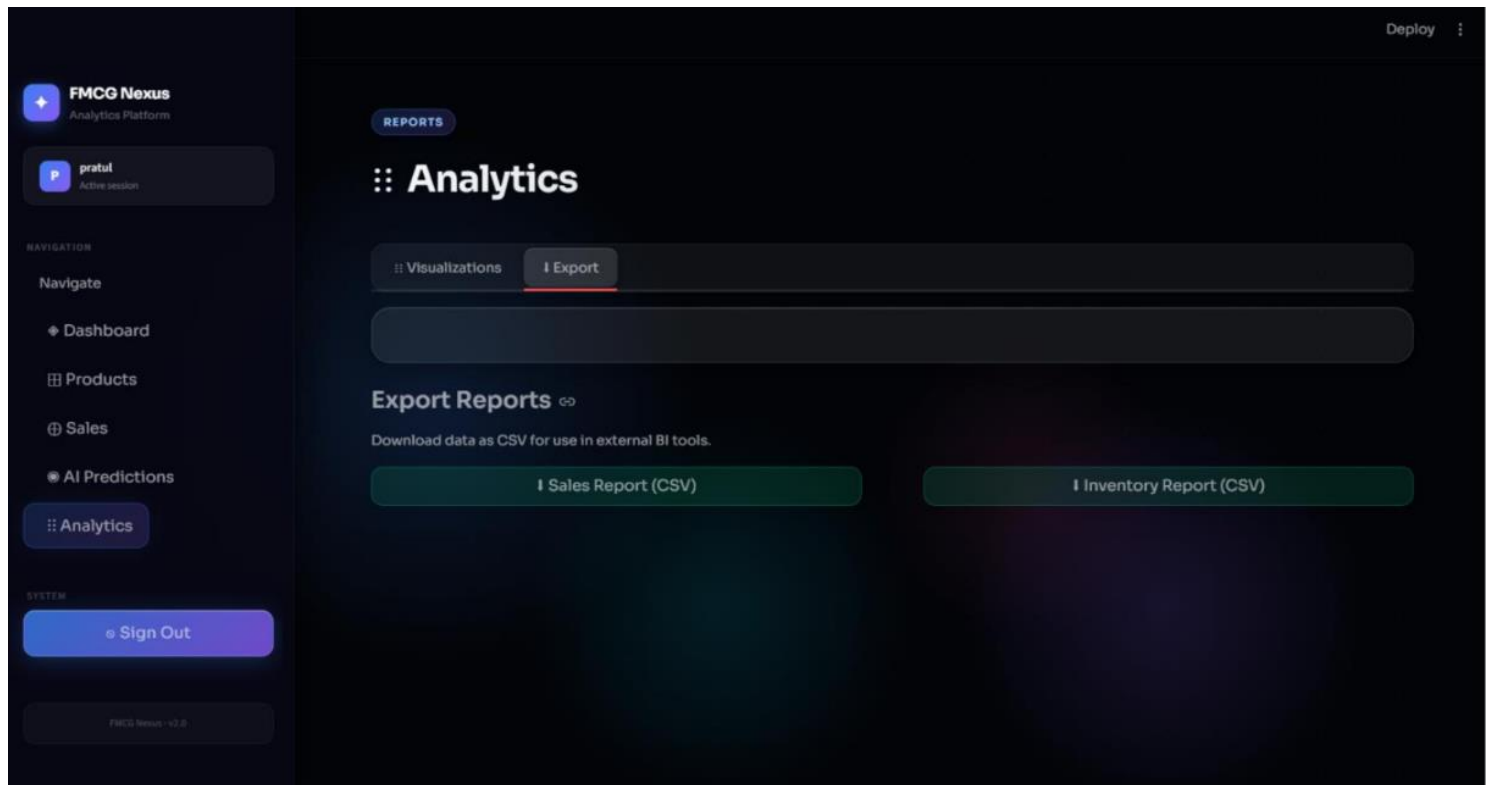
if __name__ == "__main__":
    main()
```

Output:









Result:

Thus, the FMCG Analytics and Prediction application using MongoDB as the database is developed and verified successfully, with Streamlit as the frontend interface and FastAPI (Python) as the backend, integrating machine learning models for demand and price prediction.